
Neural Dual Bounds: Valid-by-Construction JGLP Warm-Starts for MAP and Constrained MAP

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We present Neural Dual Bounds, a calibration-free framework that produces de-
2 terministic upper bounds for MAP inference in probabilistic graphical models
3 using a single neural-network forward pass. The framework trains a network to
4 output the edge messages of a mini-bucket style upper bound on a given join-graph
5 decomposition in a self-supervised manner by directly minimizing a smooth dual
6 objective. The output of this network can be used at test time to warm-start a
7 standard JGLP optimizer. Across diverse benchmarks, this warm start yields dual
8 gaps that are orders of magnitude tighter than those from zero initialization, with
9 the largest gains in the early-iteration regime. We also apply our framework for the
10 constrained MAP (CMPE) task, and we show that neural warm starts with test-time
11 adaptation consistently achieve the tightest dual gaps across all iteration budgets
12 against an adaptive baseline.

13 1 Introduction

14 Many applications of probabilistic graphical models (PGMs) repeatedly solve closely related inference
15 problems on a fixed graph. Branch-and-bound search expands one node per query, adversarial
16 robustness certification verifies one input per query, constrained planning evaluates one constraint per
17 query, and bound-based parameter learning computes a new bound at each gradient step. In these
18 settings, per-query inference often dominates wall-clock time. Classical polynomial-time dual bounds,
19 including join-graph linear programming (JGLP) [9], Max-Product Linear Programming (MPLP) [6],
20 and TRW-BP [18], provide valid upper bounds, but often require hundreds of coordinate-descent
21 iterations per query. This repeated-query structure creates a natural opportunity for amortization,
22 provided that amortization preserves the validity guarantees required by downstream algorithms.

23 Learned methods can amortize inference, but they typically sacrifice deterministic per-query validity.
24 Learned bound predictors usually provide only statistical validity: if a predicted upper bound
25 violates the true upper-bound property on even a small fraction of queries, branch-and-bound may
26 prune the optimal solution and robustness certificates may become invalid [11, 5, 10]. Conformal
27 calibration [17, 2] can reduce this risk by adding a safety margin estimated from held-out data, but
28 it still permits violations at the target residual failure rate. Learned inference networks based on
29 graph neural networks [15, 19, 20] predict approximate inference outputs, but do not enforce an
30 upper-bound certificate for every query. Existing learned methods therefore accelerate inference by
31 weakening the certificate that makes classical dual bounds useful in exact and safety-critical pipelines.

32 This validity-speed tradeoff is especially important for constrained MAP inference, which seeks the
33 most probable explanation subject to one or more budgets defined by a second log-linear model.
34 This formulation extends standard MAP inference to settings with explicit side constraints and
35 captures safety, fairness, adversarial, and resource-constrained reasoning problems [13, 12]. Existing
36 upper-bounding methods for this setting optimize a single scalar Lagrange multiplier by subgradient

descent [12]. They do not directly handle the vector-valued multipliers induced by multiple assurance requirements. With multiple constraints, projected gradient descent over the multiplier vector must solve an inner MAP-bound problem at every step, while queries may vary in evidence, costs, constraint budgets, or Lagrange multipliers. This nested optimization makes valid amortized bounds particularly valuable in the multi-constraint setting.

We introduce *Neural Dual Bounds*, a framework for amortized inference that produces deterministically valid upper bounds from a single neural-network forward pass. Instead of learning bound values directly, the network outputs antisymmetric edge messages on a join-graph decomposition. This parameterization restricts every output to a feasible subspace of the JGLP bound, so the resulting bound is valid by construction rather than by training accuracy, calibration, or a held-out quantile estimate. Antisymmetric messages preserve feasibility because each message added to one side of an edge is subtracted from the other, leaving the decomposed upperbound objective unchanged while reparameterizing local potentials. The guarantee holds for trained, untrained, and adversarial network outputs.

The same construction also provides effective warm-starts for iterative dual optimization. For MAP inference, we train the model end-to-end against a smooth objective, without an LP solver in the inner loop and without JGLP-converged labels. For constrained MAP inference, we supervise the network with LP-optimal edge messages for the per- μ Lagrangians, since this supervision is cheaper than self-supervised training to comparable tightness in this setting. We also show that an LP-free training variant can achieve similar bound quality. Thus, the framework supports both direct amortized certification and accelerated iterative tightening.

Contributions.

1. **Deterministically valid neural bounds.** We restrict the neural-network output to a structurally valid subspace of the JGLP upper bounds using an antisymmetric scatter map. This construction yields, to our knowledge, the first learned MAP upper-bound predictor whose validity is enforced by architecture rather than by post-hoc calibration.
2. **Self-supervised dual amortization without LP labels.** We train the model end-to-end against a smooth dual objective, with no LP solver in the inner training loop and no JGLP-converged labels. This objective removes the need to generate optimal targets for MAP training and makes the method applicable when LP solves are unavailable or prohibitively expensive.
3. **Extension to multi-constraint constrained MAP inference.** We apply the framework to vector-valued Lagrange multipliers using projected gradient descent, neural warm-starts, and replay-buffer test-time adaptation at each step.
4. **Empirical validation on benchmark PGMs.** On 15 UAI benchmark PGMs spanning Bayesian networks, image segmentation, and grid Markov random fields, neural warm-starts reduce the iteration-10 dual gap by up to six orders of magnitude relative to zero-initialized JGLP. On constrained MAP inference, test-time adaptation closes the gap to within 10^{-7} of the LP optimum at iteration 50 on at least 99% of Bayesian-network and segmentation queries.
5. **Structural applicability beyond JGLP.** We identify a broader class of decomposed Lagrangian duals whose message structure admits the same antisymmetric parameterization, including dual decomposition, TRW-BP, weighted CSPs, and minimum-cost flow.

2 Background

A probabilistic graphical model $(G, \mathcal{F}, \theta)$ defines a factored distribution of n discrete variables $\mathbf{x} = (x_1, \dots, x_n)$ with domains $\mathcal{D}_i$ over a graph $G = (V, E)$ with cliques $\mathcal{F}$ as

$$p(\mathbf{x} \mid \theta) \propto \prod_{\alpha \in \mathcal{F}} \exp(\theta_{\alpha}(\mathbf{x}_{\alpha})) \quad (1)$$

where $\mathcal{F}$ indexes cliques of G with scopes $\mathbf{x}_{\alpha} \subseteq \mathbf{x}$ and $\theta_{\alpha} : \mathcal{D}_{\alpha} \rightarrow \mathbb{R}$ are known as the log-potential functions.

The MAP problem is to find an assignment that maximizes the probability distribution p or, equivalently, maximizing the log probability

$$\text{MAP}^*(\theta) = \max_{\mathbf{x}} \sum_{\alpha \in \mathcal{F}} \theta_{\alpha}(\mathbf{x}_{\alpha}). \quad (2)$$

Exact MAP is NP-hard in general [16], and standard methods designed for this task, such as variable elimination, scale exponentially in the tree-width of G . As a result, a variety of approximation methods have been designed that approximately solve the MAP problem via upper bound minimization. The Mini-bucket elimination (MBE, 3), one such upper bound minimization approach, further regroups the clique potentials into clusters determined by a specified i -bound parameter. Specifically, the procedure induces a *join graph* $\mathcal{G} = (\mathcal{C}, \mathcal{E}, \theta)$ whose nodes $\mathcal{C} = \{C_1, \dots, C_K\}$ correspond to a subset of variables $\alpha \subseteq E$ with $|\alpha| \leq i + 1$ and edges in $\mathcal{E}$ connect clusters that share variables. For each edge $(\alpha, \beta) \in \mathcal{E}$, the *separator* $S_{\alpha\beta} = C_\alpha \cap C_\beta$ is the shared subset. Each cluster carries a log-potential function θ_{C_α} that aggregates the original log-potentials assigned to it during partitioning. MBE then uses the join graph clustering to produce an upper bound on the MAP value as

$$D_0 = \sum_{\alpha=1}^K \max_{\mathbf{x}_{C_\alpha}} \theta_{C_\alpha}(\mathbf{x}_{C_\alpha}) \geq \text{MAP}^*, \quad (3)$$

where the inequality comes from the observation that a maximum of a sum is always less than or equal to the sum of maximums, i.e., $\max_x (f(x) + g(x)) \leq \max_x f(x) + \max_x g(x)$.

The bound (3) is loose because adjacent clusters maximize independently and may pick conflicting values on their shared separator. *Reparameterization* tightens the bound by shifting cost between adjacent clusters via edge messages $\lambda_{\alpha\beta}(\mathbf{x}_S) \in \mathbb{R}$:

$$\theta_{C_\alpha}(\mathbf{x}_{C_\alpha}, \lambda) = \theta_{C_\alpha}(\mathbf{x}_{C_\alpha}) + \sum_{\beta \in N(\alpha)} \lambda_{\alpha\beta}(\mathbf{x}_{S_{\alpha\beta}}), \quad (4)$$

subject to an *antisymmetry constraint*

$$\lambda_{\alpha\beta}(\mathbf{x}_S) + \lambda_{\beta\alpha}(\mathbf{x}_S) = 0, \quad \forall (\alpha, \beta) \in \mathcal{E}, \forall \mathbf{x}_S. \quad (5)$$

Antisymmetry preserves the underlying joint: for any complete assignment $\mathbf{x}^*$, the message added to one cluster on a shared separator is exactly the message subtracted from its neighbor, so $\sum_\alpha \theta_{C_\alpha}(\mathbf{x}_{C_\alpha}^*, \lambda) = \sum_\alpha \theta_{C_\alpha}(\mathbf{x}_{C_\alpha}^*)$. Replacing the assignment-specific sum by per-cluster maxes can only increase the value, so the reparameterized bound

$$D(\theta, \lambda) = \sum_{\alpha=1}^K \max_{\mathbf{x}_{C_\alpha}} \theta_{C_\alpha}(\mathbf{x}_{C_\alpha}, \lambda) \geq \text{MAP}^* \quad (6)$$

holds for every λ in the antisymmetric subspace.

As $D(\theta, \lambda)$ is convex in λ (a sum of pointwise maxima of affine functions), a variety of techniques can now be applied to find the minimum of this upper bound. The JGLP [9] method minimizes it by coordinate descent. Each iteration of JGLP corresponds to one pass over the separator edges: for $(\alpha, \beta) \in \mathcal{E}$, the algorithm computes the max-marginals of C_α and C_β onto $S_{\alpha\beta}$, equalizes them to their geometric mean (the arithmetic mean in log domain), and applies the corresponding antisymmetric correction to both cluster tables (Ihler et al. [9] Algorithm 4, lines 5–8). The fixed point is the optimum $\min_\lambda D(\theta, \lambda)$, which corresponds to the optimum of a specific LP relaxation of the MAP problem on the chosen join graph.

JGLP starts from a *moment-matched* mini-bucket join graph rather than raw MBE: at construction, one round of cost-shifting on the mini-buckets within each elimination bucket equalizes their max-marginals on the shared scope (Ihler et al. [9] Algorithm 5, “MBE-MM”). This produces a slightly tighter starting point than (3) before any JGLP iterations have run. Throughout this paper, “ $\lambda = \mathbf{0}$ ” denotes the JGLP coordinate-descent state at iteration zero *after* MBE-MM construction, matching the convention of all JGLP implementations we benchmark against.

3 Method

Reparameterizations of a join-graph decomposition [9, 18] are constrained to lie in the antisymmetric subspace defined by (5); any element of this subspace yields a valid bound. Existing methods (JGLP, MPLP, TRW-BP) navigate this subspace iteratively. However, convergence of JGLP often requires hundreds of iterations on the benchmark models we consider, with each iteration costing $\mathcal{O}(|\mathcal{E}| \cdot d^{i+1})$, where d is the maximum domain size. As a result, a poor initialization can be far from the tightest

upper bound. This motivates our approach in which we train a neural network to directly predict the λ s from a given partial assignment to the random variables. This approach is particularly applicable in typical scenarios in which a PGM has already been fit to data and we are simply using the PGM to make (repeated) MAP inference queries.

A few remarks, first, the complexity of computing the upper bounds in MBE scales exponentially in the i -bound. While this is polynomial time computable for fixed i , it is far from practical on large networks. Our motivation then is to train a neural network for this polynomial time computable task. The aim is to move the computational cost from test time to training time. As the bound is polynomial time computable for fixed i , there is some hope that a sufficiently complicated neural network with a reasonable number of training examples could well-approximate the iterative methods. If our approach is successful, the cost of computing a “good” upper bound at test time will correspond only to the cost of evaluating our neural network for a fixed set of evidence.

To efficiently encode the upper bound prediction task in a neural network architecture, we make a few small changes to the representation of the MAP prediction task.

First, we will train our neural network by directly minimizing the dual objective. So, no labeled training examples are required. This makes the approach flexible across many different models and does not require us to compute expensive i -bounds during training. To simplify training, we replace the hard max in the upper bound with a smooth log-sum-exp surrogate.

$$D_\tau(\theta, \lambda) = \sum_{\alpha} \tau \log \sum_{\mathbf{x}_{C_\alpha}} \exp \left(\frac{\theta_{C_\alpha}(\mathbf{x}_{C_\alpha}, \lambda)}{\tau} \right), \quad (7)$$

where $\tau > 0$ is a temperature. As $\tau \rightarrow 0$, $D_\tau \rightarrow D$.

Proposition 1. $D_\tau(\theta, \lambda) \geq D(\theta, \lambda) \geq \text{MAP}^*$ for all $\tau > 0$ and all λ .

$D(\theta, \lambda)$ is convex in λ (sum of pointwise maxima of affine functions), so subgradient methods could be used directly, but we use the smooth surrogate D_τ for compatibility with standard gradient-based optimizers.

Prior amortized-inference work either supervises against a slower exact solver [11] or unrolls iterations of the target algorithm and backpropagates through them. We do neither: D_τ is a closed-form differentiable function, so a single backward pass through the smooth dual yields the gradient. The training distribution is evidence configurations sampled at varying observation ratios, with no inference labels of any kind. This makes training 50–200 $\times$ cheaper than supervised alternatives and removes LP-solver availability as a deployment requirement.

Second, we represent λ more compactly. The antisymmetry constraint (5) couples the two directions of every edge, so it suffices to store one real number per (undirected-edge, separator-assignment) pair. Stack these into a single edge-message vector $m \in \mathbb{R}^{\text{edim}}$, where

$$\text{edim} = \sum_{(\alpha, \beta) \in \mathcal{E}} \prod_{i \in S_{\alpha\beta}} d_i$$

counts those pairs and d_i is the domain size of variable i . Let $\lambda_{\text{dim}} = \sum_{\alpha} \prod_{i \in C_\alpha} d_i$ be the total number of cluster-table entries, and let $\theta_{\text{flat}} \in \mathbb{R}^{\lambda_{\text{dim}}}$ be the concatenation of all cluster tables, ordered so that the α -th block of size $\prod_{i \in C_\alpha} d_i$ holds the entries $\{\theta_{C_\alpha}(x_{C_\alpha})\}_{x_{C_\alpha}}$ of cluster α . The reparameterization is then a single linear map

$$\theta'_{\text{flat}} = \theta_{\text{flat}} + A m,$$

where $A \in \{-1, 0, +1\}^{\lambda_{\text{dim}} \times \text{edim}}$ is a fixed sparse matrix determined entirely by the join-graph structure. Each column of A corresponds to one (edge, separator-assignment) pair $((\alpha, \beta), x_S)$ and contains $+1$ in every row of θ_{flat} that lies in cluster α and is consistent with x_S , -1 in every row that lies in cluster β and is consistent with x_S , and 0 elsewhere. The matched ± 1 pattern makes every column of A sum to zero, which is exactly the antisymmetry of the reparameterization (5).

As the neural network essentially produces λ s that satisfy the antisymmetry constraint, its output can always be turned into a valid upper bound on the MAP problem. Consequently, training never has to enforce feasibility as a constraint or check it post-hoc; the architecture restricts predictions to the valid set. This is structurally different from calibration-based methods, which produce predictions in an unrestricted space and use a held-out calibration set to estimate the residual violation probability.

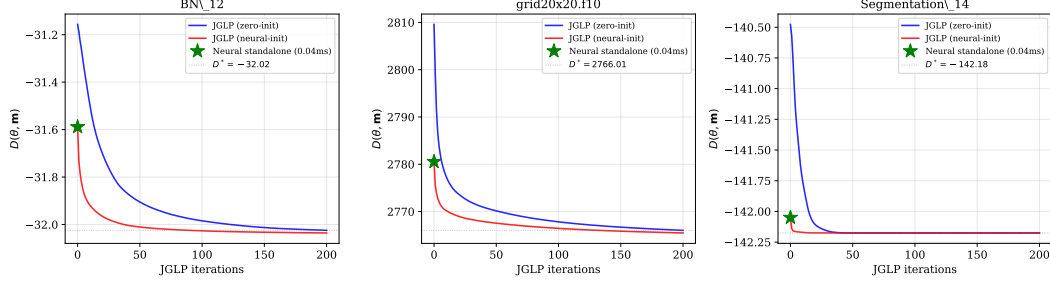


Figure 1: **JGLP convergence with and without neural warm-start** on three representative PGMs (BN_12, grid20x20.f10, Segmentation_14). Both curves provably converge to the same optimum D^* since $D(\theta, \mathbf{m})$ is convex in $\mathbf{m}$. The green star marks the standalone neural bound from a single forward pass (~ 0.04 ms) and is used as the warm-start for JGLP at iteration 0; it starts strictly below zero-init and reaches a small neighborhood of D^* in fewer JGLP iterations. The three panels span the BN, Grid, and Segmentation families; full results across all 37 PGMs are reported in Appendix B.2 and Table 1.

Finally, note that the upper bounds produced by our methods and the iterative methods in the best case yield the tightest upper bound on MAP* attainable by the given join graph decomposition, but this bound is tight only in special cases, e.g., such as tree-structured models or log-supermodular models [14].

4 Experiments

We evaluate on 15 PGMs from the UAI 2022 inference competition, covering Bayesian Networks, Image Segmentation, Grid Markov Random Fields. Table 1 (MAP) and Tables 2–3 ($K=2$ CMPE); full results for $K=1$ CMPE are in Appendix B.17. All experiments use i -bound 4 and 10 random seeds per (PGM, evidence-bin) cell. Our reference D^{LP} is the exact LP optimum solved with CVXPY/CLARABEL (Appendix B.1); The JGLP coordinate descent occasionally terminates at non-globally-optimal fixed points, so we use the LP optimum rather than JGLP’s converged value as the ground-truth lower bound on the dual.

We consider two sets of experiments. The first experiment explores how well the trained neural network does at finding good values for the messages for a single MAP estimation task. In the second set of experiments we apply the same strategy to approximately solve constrained most probable explanation tasks.

4.1 Neural Network Training vs. JGLP

The primary evaluation of our neural bounds is how much iterative inference they save. A single neural forward pass (~ 0.04 ms) produces a valid upper bound on MAP*; this bound is typically tighter than the MBE starting point and can serve as a warm-start for JGLP. We measure the effective speedup by counting JGLP iterations needed to reach a target relative error from the converged dual D^* , comparing zero-init vs neural-init.

We use a 3-layer MLP with LayerNorm and GELU activations: $512 \rightarrow 512 \rightarrow 256 \rightarrow e_{\text{dim}}$. The output layer has no activation function (edge messages are unconstrained). The output is initialized with small random weights (std = 0.001) so that the initial bound $D(f_\phi(\mathbf{e})) \approx D_0$ (the MBE bound).

We anneal τ , the temperature parameter of the smooth surrogate, during training: $\tau = 0.01$ for the first third, then $\tau = 0.001$ for the remaining epochs. We use AdamW optimization with cosine annealing (lr = 10^{-3} , 200 epochs, batch size 64). Training data consists of evidence configurations sampled at various evidence ratios (10–90% of variables observed). No JGLP or other iterative inference is required during training.

Since $D(\theta, \mathbf{m})$ is a convex function of $\mathbf{m}$, we can compute the exact optimum using via CVXPY/CLARABEL (formulation in Appendix B.1). Table 1 reports the absolute gap $D^{(t)} - D^{LP}$

Table 1: **Standalone bound quality**: dual gap $|D - D^{\text{LP}}|$ (mean $\pm$ std) at fixed JGLP iterations. J = zero-init, N = neural-init. Mean and population standard deviation over 10 bin-0 test seeds per PGM.

PGM	iter=10		iter=50		iter=100	
	J	N	J	N	J	N
BN_12	0.638 $\pm$ 0.238	0.373 $\pm$ 0.179	0.184 $\pm$ 0.147	0.116 $\pm$ 0.105	0.088 $\pm$ 0.097	0.060 $\pm$ 0.072
BN_13	0.474 $\pm$ 0.115	0.255 $\pm$ 0.088	0.116 $\pm$ 0.073	0.088 $\pm$ 0.072	0.058 $\pm$ 0.053	0.050 $\pm$ 0.054
BN_14	0.278 $\pm$ 0.173	0.174 $\pm$ 0.128	0.094 $\pm$ 0.137	0.042 $\pm$ 0.037	0.055 $\pm$ 0.143	5.75e-03 $\pm$ 8.00e-03
BN_9	0.162 $\pm$ 0.156	0.061 $\pm$ 0.070	0.126 $\pm$ 0.169	6.80e-03 $\pm$ 0.019	0.125 $\pm$ 0.170	1.47e-03 $\pm$ 4.42e-03
Seg_12	1.19 $\pm$ 0.900	0.701 $\pm$ 0.761	0.177 $\pm$ 0.321	0.019 $\pm$ 0.035	0.163 $\pm$ 0.327	1.96e-07 $\pm$ 1.11e-07
Seg_14	0.189 $\pm$ 0.047	0.018 $\pm$ 0.016	2.20e-06 $\pm$ 3.03e-06	1.47e-04 $\pm$ 2.47e-04	2.20e-06 $\pm$ 3.03e-06	6.06e-06 $\pm$ 8.42e-06
Seg_16	1.93 $\pm$ 0.396	0.417 $\pm$ 0.273	0.236 $\pm$ 0.202	0.029 $\pm$ 0.075	0.028 $\pm$ 0.057	7.66e-06 $\pm$ 1.50e-05
Seg_18	0.627 $\pm$ 0.692	0.419 $\pm$ 0.478	0.073 $\pm$ 0.102	0.050 $\pm$ 0.077	5.06e-03 $\pm$ 0.015	4.45e-06 $\pm$ 9.49e-06
Seg_19	0.125 $\pm$ 0.150	0.104 $\pm$ 0.114	0.044 $\pm$ 0.123	6.82e-03 $\pm$ 0.012	0.044 $\pm$ 0.123	5.24e-06 $\pm$ 6.52e-06
grid10 $\times$ 10.f10	9.61e-02 $\pm$ 1.43e-02	3.29e-02 $\pm$ 1.06e-02	1.27e-10 $\pm$ 1.19e-10	4.9e-04 $\pm$ 4.73e-05	1.27e-10 $\pm$ 1.19e-10	4.49e-05 $\pm$ 3.41e-05
grid10 $\times$ 10.f5	9.12e-09 $\pm$ 1.17e-08	1.97e-03 $\pm$ 2.06e-03	9.12e-09 $\pm$ 1.17e-08	1.22e-05 $\pm$ 1.73e-05	9.12e-09 $\pm$ 1.17e-08	4.61e-08 $\pm$ 2.61e-08
grid20 $\times$ 20.f5	1.33 $\pm$ 0.849	1.30 $\pm$ 0.850	0.117 $\pm$ 0.115	0.115 $\pm$ 0.114	9.90e-03 $\pm$ 0.023	9.22e-03 $\pm$ 0.021
Grids_26	3.13 $\pm$ 2.19	3.12 $\pm$ 2.17	0.138 $\pm$ 0.147	0.138 $\pm$ 0.147	2.51e-08 $\pm$ 4.86e-08	8.62e-07 $\pm$ 9.57e-07
Grids_28	5.21 $\pm$ 3.22	5.22 $\pm$ 3.19	0.279 $\pm$ 0.383	0.279 $\pm$ 0.380	9.18e-09 $\pm$ 1.32e-08	2.65e-06 $\pm$ 1.92e-06
Grids_29	1.33 $\pm$ 0.849	1.30 $\pm$ 0.851	0.117 $\pm$ 0.115	0.115 $\pm$ 0.114	9.90e-03 $\pm$ 0.023	9.23e-03 $\pm$ 0.021

at fixed JGLP iteration checkpoints $t \in \{10, 50, 100\}$, mean $\pm$ std over 10 bin-0 seeds per PGM. Lower is better.

Across the 15 PGMs, neural warm-start matches or beats zero-init at iter=10 on 100% of seeds for 11 of them. The gap then closes rapidly as JGLP iterates: on the Segmentation family it drops by 4–6 orders of magnitude by iter=100—for example, Seg_12’s zero-init gap falls from 0.701 to 2×10^{-7} —while BN models show smaller but consistent gains (~ 1 –2 orders). Three regimes resist this pattern. (i) *Degenerate grids* (grid10 $\times$ 10, grid40 $\times$ 40.f2) already sit within 10^{-8} of D^{LP} at MBE construction with i -bound 4; neural cannot improve, and the small overshoot at iter=10 is floating-point round-off, not a training failure. (ii) *Promedas networks* have near-zero dual values, so absolute gaps look small even when relative gains are substantial. (iii) *Very-low-evidence Seg_19* converges fast enough on its own that zero-init reaches 10^{-3} within 10 iterations; neural overshoots briefly before both methods agree on the typical seed by iter=100, though linear-mean variance remains high. The ~ 0.04 ms forward pass is recovered after saving a single JGLP iteration, so the warm-start is net-positive whenever JGLP runs.

Amortized cost analysis. Neural forward pass costs 0.03–1.3ms regardless of PGM size. A single JGLP iteration costs 1.6–131ms depending on PGM complexity. The breakeven point (where amortized neural cost equals JGLP cost) is 5–20 queries. For applications requiring ≥ 20 queries on the same graph structure (e.g., parameter learning inner loops, constrained MAP search), the neural approach is strictly dominant.

Edge vs cluster parameterization. The edge-message parameterization (e_{dim}) is $2 \times$ smaller than the cluster-table parameterization (λ_{dim}) while capturing 98.6% of JGLP’s corrections (ltsq residual 1.4%). The reduced output dimension improves generalization: self-supervised training with edge messages achieves 94% neural-better vs. 0% with direct cluster-table prediction (which violates the reparameterization constraint).

Self-supervised vs supervised training. We compare pure self-supervised training (directly minimizing D_{τ}) against supervised training with JGLP edge-message targets. On BN_12 bin 0: self-supervised achieves 94% neural-better vs 72% for MSE-supervised. The MSE model produces larger magnitude edge shifts ($2.8 \times$ scale ratio) but sometimes overshoots the optimal reparameterization, resulting in 28% of instances worse than zero-init. The self-supervised loss naturally avoids this by directly optimizing the bound quality.

4.2 Constrained Most Probable Explanation

Rahman et al. [12] consider a constrained version of the most probable explanation task that allows constraints that are given by PGMs that, for simplicity, are over the same graph and factorization. Formally, let $(G, \mathcal{F}, \theta)$ be a PGM. Consider C constraint PGMs $(G, \mathcal{F}, \theta^c)$ for each $c \in \{1, \dots, C\}$ over the same graph and factorization with possibly different potential functions. Each constraint will

239 be of the form $\sum_{\alpha \in \mathcal{F}} \theta_{\alpha}^c(x_{\alpha}) \leq q_c$ for some $q_c \in \mathbb{R}$. This yields the optimization problem

$$\begin{aligned} & \max_x \sum_{\alpha \in \mathcal{F}} \theta_{\alpha}(x) \text{ subject to} \\ & \sum_{\alpha \in \mathcal{F}} \theta_{\alpha}^c(x_{\alpha}) \leq q_c \text{ for each } c \in \{1, \dots, C\}. \end{aligned}$$

240 Note that we can apply the same argument to the case where each one of the PGMs is replaced by
 241 the same join graph (again with possibly different log-factors). To handle constrained optimization
 242 problems of this form, Rahman et al. [12] consider a Lagrangian relaxation

$$\begin{aligned} \mathcal{L}(\mu) &= \max_x \left[\sum_{\alpha \in \mathcal{F}} \theta_{\alpha}(x) - \sum_c \mu_c \left(\sum_{\alpha \in \mathcal{F}_c} \theta_{\alpha}^c(x_{\alpha}) - q_c \right) \right] \\ &\leq \sum_{\alpha \in \mathcal{F}} \max_x \left[\theta_{\alpha}(x) - \sum_c \mu_c \theta_{\alpha}^c(x_{\alpha}) \right] - \sum_c \mu_c q_c. \end{aligned}$$

243 Since the lower bound on $\mathcal{L}$ is convex in $\mu \geq 0$, we can minimize it via projected (sub)gradient
 244 descent. To compute the subgradients, we must compute the max in the lower bound, which can be
 245 done (approximately) with JGLP. As a result, we can train a μ -conditioned neural model for this task:
 246 the input is (evidence_encoding, μ) and the output is edge messages $\mathbf{m}$. Training data consists of
 247 100 evidence configurations $\times$ 50 values of $\mu \in [0, 3]$, yielding 5,000 samples per PGM.

248 Following [12], we construct each CMPE instance by perturbing the log-potentials of the original
 249 model: for every clique α and assignment x_{α} , we draw $\epsilon \sim \mathcal{N}(0, \sigma^2)$ i.i.d. per table entry and set
 250 $\theta_{\alpha}^c(x_{\alpha}) = \theta_{\alpha}(x_{\alpha}) + \epsilon$. The constraint threshold q is the 80th percentile of $\sum_{\alpha} \theta_{\alpha}^c(x_{\alpha})$ on the test
 251 sample. We use $\sigma = 0.5$ to keep the constraint active at the optimal multiplier ($\mu^* > 0$); with $\sigma = 0.1$
 252 we have $\theta^c \approx \theta$, so at $\mu^* \approx 1$ the combined Lagrangian $\theta - \mu^* \theta^c$ collapses to pure noise.

253 We compare four different methods to estimate the subgradients: (1) **JGLP**—initialized at $\mathbf{m} = 0$; (2)
 254 **NN**—inject NN-predicted $\mathbf{m}$ then run JGLP; (3) **JGLP+WS**—warm-start JGLP from $\mathbf{m}$ given by the
 255 most recent run of JGLP; (4) **NN+TTA**—test-time adaptation, at each gradient step k , fine-tune the
 256 NN on all previous $(\mu_j, \theta^{(\mu_j)})$ pairs via the self-supervised D_{τ} loss (replay buffer of up to 20 entries,
 257 10 inner gradient steps). By step 15+ of test time adaptation, the replay buffer contains supervised
 258 signal from the local μ trajectory, sharpening predictions. The validity guarantee $L(\mu) \leq \text{CMPE}^*$
 259 holds for any $\mathbf{m}$.

260 We use a generic test-time adaptation (TTA) mechanism: we store recent (input feature, converged
 261 dual variable, log-Z) tuples in a replay buffer of capacity $|R| = 20$, and after each query fine-tune the
 262 model for 10 inner gradient steps on D_{τ} over the buffer. The buffer is FIFO; the loss is identical to
 263 the offline objective, so no separate test-time validation is required and the buffer accumulates no
 264 out-of-distribution examples by construction.

265 **Results.** We run 100 gradient steps on μ and at each step run JGLP from each initialization for up
 266 to 500 iterations (or until $|D(t) - D(t-1)| < 10^{-6}$). Table 18 reports the dual gap ($L(\mu) - L^{\text{LP}}(\mu)$)
 267 at fixed inner JGLP iteration checkpoints, with median computed over 100 μ -grad steps $\times$ 10 test
 268 evidence seeds.

269 Test-time replay adaptation consistently drives the dual gap ($L(\mu) - L^{\text{LP}}(\mu)$) closer to zero by 1–4
 270 orders of magnitude vs zero-init at iter=10, and up to 5 orders of magnitude by iter=50. The BN and
 271 Segmentation models show strong gains (78–100% win@10); Grids models also benefit (92–95%);
 272 grid10 $\times$ 10 is degenerate (MBE already at L^{LP} , zero-init starts within 10^{-8} of LP). Even offline NN
 273 warm-start (M2) helps on most PGMs, with M3 additionally specializing to the local μ trajectory via
 274 the replay buffer.

275 JGLP+WS (no learning, just carrying converged $\Delta\lambda$ across μ -steps) is the strongest non-neural
 276 baseline. On Bayesian networks (BN_9, BN_12, BN_13, BN_14), μ -trajectories converge quickly
 277 and successive optimal λ values are nearby, so λ -continuity alone closes most of the gap (e.g., BN_12
 278 iter=10: JGLP+WS 0.020 vs NN+TTA 0.063). On grids and segmentation models, μ keeps moving
 279 across the trajectory and stale $\Delta\lambda$ pulls JGLP toward the previous optimum; NN+TTA’s learned
 280 structural prior dominates by 1–3 orders of magnitude (Seg_14 iter=10: JGLP+WS 6.40 vs NN+TTA

Table 2: **CMPE bound quality** (two constraints, single-pass methods): absolute dual gap $|L(\mu) - L^{\text{LP}}(\mu)|$ (mean $\pm$ std).

PGM	iter=10		iter=50		iter=100	
	JGLP	NN	JGLP	NN	JGLP	NN
BN_12	0.658 $\pm$ 0.149	0.646 $\pm$ 0.149	0.222 $\pm$ 0.072	0.218 $\pm$ 0.066	0.111 $\pm$ 0.060	0.109 $\pm$ 0.051
BN_13	0.488 $\pm$ 0.163	0.436 $\pm$ 0.097	0.179 $\pm$ 0.240	0.097 $\pm$ 0.054	0.125 $\pm$ 0.255	0.036 $\pm$ 0.035
BN_14	0.352 $\pm$ 0.083	0.313 $\pm$ 0.086	0.049 $\pm$ 0.058	0.041 $\pm$ 0.036	0.017 $\pm$ 0.052	9.70e-03 $\pm$ 0.014
BN_9	0.190 $\pm$ 0.083	0.161 $\pm$ 0.084	0.101 $\pm$ 0.093	0.043 $\pm$ 0.049	0.072 $\pm$ 0.100	0.011 $\pm$ 0.023
Seg_12	0.592 $\pm$ 0.505	0.597 $\pm$ 0.508	0.047 $\pm$ 0.085	0.046 $\pm$ 0.083	1.52e-03 $\pm$ 3.03e-03	1.48e-03 $\pm$ 2.91e-03
Seg_14	0.712 $\pm$ 0.879	0.620 $\pm$ 0.756	4.72e-03 $\pm$ 0.012	3.28e-03 $\pm$ 8.60e-03	2.03e-05 $\pm$ 7.47e-05	1.96e-05 $\pm$ 6.46e-05
Seg_16	2.37 $\pm$ 2.06	2.05 $\pm$ 1.78	0.234 $\pm$ 0.325	0.207 $\pm$ 0.280	0.023 $\pm$ 0.050	0.019 $\pm$ 0.041
Seg_18	0.745 $\pm$ 0.765	0.721 $\pm$ 0.720	0.030 $\pm$ 0.059	0.020 $\pm$ 0.050	3.25e-03 $\pm$ 0.012	3.96e-05 $\pm$ 7.71e-05
Seg_19	2.81 $\pm$ 3.83	2.54 $\pm$ 3.53	0.917 $\pm$ 1.36	0.841 $\pm$ 1.31	0.436 $\pm$ 0.837	0.393 $\pm$ 0.808
grid10x10.f10	0.096 $\pm$ 0.096	1.94 $\pm$ 0.519	5.85e-11 $\pm$ 5.47e-11	0.040 $\pm$ 0.023	5.85e-11 $\pm$ 5.47e-11	8.79e-04 $\pm$ 8.79e-04
grid10x10.f5	0.434 $\pm$ 0.397	0.182 $\pm$ 0.213	0.406 $\pm$ 0.420	1.90e-05 $\pm$ 3.62e-05	0.406 $\pm$ 0.420	2.75e-07 $\pm$ 5.64e-07
grid20x20.f5	12.90 $\pm$ 18.02	12.90 $\pm$ 18.02	1.54 $\pm$ 2.43	1.54 $\pm$ 2.43	0.060 $\pm$ 0.173	0.060 $\pm$ 0.173
Grids_26	49.58 $\pm$ 65.71	49.59 $\pm$ 65.70	3.60 $\pm$ 7.09	3.60 $\pm$ 7.09	0.124 $\pm$ 0.303	0.124 $\pm$ 0.303
Grids_28	109.22 $\pm$ 139.60	109.23 $\pm$ 139.60	6.95 $\pm$ 16.17	6.95 $\pm$ 16.17	0.871 $\pm$ 2.37	0.871 $\pm$ 2.37
Grids_29	12.90 $\pm$ 18.02	12.90 $\pm$ 18.02	1.54 $\pm$ 2.43	1.54 $\pm$ 2.43	0.060 $\pm$ 0.173	0.060 $\pm$ 0.173

Table 3: **CMPE bound quality** (two constraints, cross- μ -step methods): absolute dual gap $|L(\mu) - L^{\text{LP}}(\mu)|$ (mean $\pm$ std).

PGM	iter=10		iter=50		iter=100	
	JGLP+WS	NN+TTA	JGLP+WS	NN+TTA	JGLP+WS	NN+TTA
BN_12	0.063 $\pm$ 0.090	0.079 $\pm$ 0.132	0.024 $\pm$ 0.036	0.037 $\pm$ 0.054	0.016 $\pm$ 0.026	0.023 $\pm$ 0.033
BN_13	0.059 $\pm$ 0.112	0.067 $\pm$ 0.113	0.011 $\pm$ 0.032	0.019 $\pm$ 0.037	3.78e-03 $\pm$ 0.015	6.51e-03 $\pm$ 0.017
BN_14	0.080 $\pm$ 0.097	0.048 $\pm$ 0.096	6.27e-03 $\pm$ 0.025	8.65e-03 $\pm$ 0.026	2.03e-03 $\pm$ 0.015	1.94e-03 $\pm$ 9.46e-03
BN_9	0.027 $\pm$ 0.046	0.028 $\pm$ 0.056	4.12e-03 $\pm$ 0.025	6.94e-03 $\pm$ 0.022	1.89e-03 $\pm$ 0.024	1.30e-03 $\pm$ 6.25e-03
Seg_12	6.09 $\pm$ 3.57	0.177 $\pm$ 0.332	0.422 $\pm$ 0.511	0.015 $\pm$ 0.048	0.075 $\pm$ 0.186	5.25e-04 $\pm$ 1.62e-03
Seg_14	9.02 $\pm$ 5.23	0.098 $\pm$ 0.321	0.283 $\pm$ 0.299	5.29e-04 $\pm$ 2.97e-03	7.13e-03 $\pm$ 0.026	5.40e-06 $\pm$ 2.37e-05
Seg_16	8.66 $\pm$ 4.79	0.477 $\pm$ 0.980	0.541 $\pm$ 0.517	0.053 $\pm$ 0.143	0.048 $\pm$ 0.081	3.68e-03 $\pm$ 0.017
Seg_18	6.95 $\pm$ 3.58	0.149 $\pm$ 0.380	0.250 $\pm$ 0.367	3.65e-03 $\pm$ 0.021	0.023 $\pm$ 0.057	5.83e-05 $\pm$ 2.29e-04
Seg_19	12.46 $\pm$ 8.61	0.889 $\pm$ 1.85	1.38 $\pm$ 1.58	0.415 $\pm$ 0.860	0.493 $\pm$ 0.967	0.251 $\pm$ 0.579
grid10x10.f10	39.09 $\pm$ 36.98	0.060 $\pm$ 0.224	5.67 $\pm$ 5.73	2.05e-03 $\pm$ 3.53e-03	1.53 $\pm$ 1.56	2.10e-04 $\pm$ 3.62e-04
grid10x10.f5	2.48 $\pm$ 2.55	0.038 $\pm$ 0.118	0.012 $\pm$ 0.031	2.72e-05 $\pm$ 7.54e-05	1.93e-04 $\pm$ 5.69e-04	5.05e-07 $\pm$ 1.38e-06
grid20x20.f5	101.50 $\pm$ 77.74	6.25 $\pm$ 11.88	3.97 $\pm$ 2.67	0.664 $\pm$ 1.56	0.213 $\pm$ 0.317	0.013 $\pm$ 0.074
Grids_26	447.14 $\pm$ 360.20	39.54 $\pm$ 55.87	14.08 $\pm$ 9.55	2.86 $\pm$ 6.57	0.827 $\pm$ 1.09	0.055 $\pm$ 0.176
Grids_28	1044.89 $\pm$ 857.50	97.37 $\pm$ 129.23	30.23 $\pm$ 22.57	6.37 $\pm$ 15.78	2.61 $\pm$ 3.45	0.838 $\pm$ 2.32
Grids_29	101.50 $\pm$ 77.74	6.37 $\pm$ 11.97	3.97 $\pm$ 2.67	0.682 $\pm$ 1.57	0.213 $\pm$ 0.317	0.014 $\pm$ 0.076

0.096; Grids_28: 436.6 vs 19.2). The two adaptation channels are complementary: λ -continuity exploits local smoothness in the optimization landscape, while the learned prior captures global structure that holds across the trajectory. We leave a hybrid (carrying $\Delta\lambda$ and TTA-fine-tuning the network) to future work.

Test-Time Adaptation The offline training distribution for the neural network sampled $\mu \sim \text{unif}(0, 1)$, but the projected gradient descent on μ at test time is unconstrained above and routinely drives μ outside this range: across all 15 PGMs, 36% of $K=1$ queries and 25% of $K=2$ queries occur at $\mu > 1$, with maxima of 36.1 and 19.4 respectively. A frozen offline-trained network on these tails would predict edge messages fitted to a different Lagrangian; deterministic validity (Prop. 1) still holds, but tightness degrades. Replay-buffer TTA continually re-fits the network to the local μ -region the trajectory has actually visited, recovering tightness within ~ 50 inner JGLP iterations even on tails 20–36 $\times$ the training upper bound (Tables 2–3). This explains the large NN+TTA-vs-NN gap on grids and segmentations, where μ keeps moving across the trajectory: NN+TTA’s primary job is online distribution adaptation, not just specialization.

5 Related Work

Learning to branch / Learning for combinatorial optimization. A growing literature uses neural networks to guide search in combinatorial optimization [11, 5, 10]. These methods typically learn heuristics (branching decisions, node selection) rather than bounds, and provide no formal guarantees on bound validity. Concurrently, [7] learn cutting sets for integer programming, but require problem-specific cut generation. Our approach is orthogonal: we learn *dual bounds* rather than search heuristics, with deterministic validity guarantees.

Amortized optimization. Amortized optimization [1] trains networks to predict solutions to optimization problems, trading per-instance computation for upfront training cost. Our work falls in this category but with a crucial difference: the “feasibility” of our predictions (validity of the bound) is guaranteed by the architecture, not learned. This contrasts with approaches that learn feasible solutions and must verify post-hoc [4].

Differentiating from RL for MILP. The RL-for-MILP literature [11, 10] learns search heuristics (branching, node selection) via reinforcement learning, providing no formal bound guarantees. Our approach differs in three key ways: (1) we learn *bounds* rather than heuristics, via self-supervised optimization (not RL); (2) we exploit PGM-specific reparameterization structure (antisymmetric messages) that has no analog in generic MILP; and (3) our bounds are *deterministically* valid by construction, not statistically calibrated. The deterministic guarantee is the core contribution: a learned bound that occasionally violates $D \geq \text{MAP}^*$ is useless for pruning, while ours is safe by design.

6 Conclusion

On UAI benchmark PGMs spanning Bayesian networks, image segmentation, and grid Markov random fields, neural warm-start reduces the iteration-10 dual gap by up to six orders of magnitude over zero-init JGLP (Table 1) confirms the same trend across protein folding and Promedas medical-diagnosis networks. On constrained MAP with one or two constraints, neural warm-start with replay-buffer test-time adaptation reaches the LP optimum to within 10^{-7} at iteration 50 on $\geq 99\%$ of (seed, μ -step) pairs for the Bayesian-network and segmentation families (Tables 2–3; full $K=1$ in Appendix B.17).

The framework extends to any Lagrangian dual problem whose decomposition admits antisymmetric messages, including dual decomposition, TRW-BP, weighted CSPs, and minimum-cost flow. The approach also extends to non-PGM problems with similar dual structure. (i) *SVM/SMO*: the SVM dual is concave in Lagrange multipliers α and linear in the training data—structurally identical to $D(\theta, \lambda)$. A neural network predicting warm-start α values would yield valid dual bounds by construction, accelerating sequential minimal optimization (SMO) in online learning settings where many SVMs are trained on similar data. (ii) *Network flow*: the LP dual of minimum-cost flow has antisymmetric dual variables on edges—directly analogous to our edge-message parameterization—so the same antisymmetric scatter matrix A applies. (iii) *Weighted CSPs*: our benchmark suite already includes weighted constraint satisfaction problems (wcsp_11, wcsp_12, wcsp_15; see appendix), demonstrating applicability beyond Bayesian networks and Markov random fields. In each case, the network architecture and self-supervised loss are identical—only the matrix A changes to reflect the problem-specific dual structure.

Our results immediately suggest two directions for future work. First, JGLP+WS (λ -continuity, no learning) and NN+TTA (learning, no λ -continuity) dominate on disjoint families in Tables 2–3: λ -continuity wins on Bayesian networks where successive μ -step optima are close, NN+TTA wins on grids and segmentations where the μ -trajectory drifts. Carrying $\Delta\lambda$ across μ -steps and fine-tuning the network combines two complementary adaptation channels and should dominate either alone, particularly when TTA must absorb out-of-distribution μ .

Second, Our MLP is trained per-graph: a new join-graph topology requires retraining. A graph neural network operating on the join graph, with edge messages produced by edge-level readouts, would amortize training across PGM topologies and remove the per-graph cost. The antisymmetric scatter A is determined entirely by the join-graph structure, so much of the approach transfers without modification to a GNN parameterization.

References

- [1] Brandon Amos. Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732, 2023.
- [2] Anastasios N Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023.

- [3] Rina Dechter and Irina Rish. Mini-buckets: A general scheme for bounded inference. *Journal of the ACM (JACM)*, 50(2):107–153, 2003.
- [4] Priya L Donti, David Rolnick, and J Zico Kolter. Dc3: A learning method for optimization with hard constraints. *arXiv preprint arXiv:2104.12225*, 2021.
- [5] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems*, 32, 2019.
- [6] Amir Globerson and Tommi Jaakkola. Fixing max-product: Convergent message passing algorithms for map lp-relaxations. *Advances in neural information processing systems*, 20, 2007.
- [7] Zeren Huang, Kerong Wang, Furui Liu, Hui-Ling Zhen, Weinan Zhang, Mingxuan Yuan, Jianye Hao, Yong Yu, and Jun Wang. Learning to select cuts for efficient mixed-integer programming. *Pattern Recognition*, 123:108353, 2022.
- [8] Christian Igel. Smooth min-max monotonic networks. *arXiv preprint arXiv:2306.01147*, 2023.
- [9] Alexander T Ihler, Natalia Flerova, Rina Dechter, and Lars Otten. Join-graph based cost-shifting schemes. *arXiv preprint arXiv:1210.4878*, 2012.
- [10] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. *Advances in neural information processing systems*, 30, 2017.
- [11] Vinod Nair, Sergey Bartunov, Felix Gimeno, Ingrid Von Glehn, Pawel Lichocki, Ivan Lobov, Brendan O’Donoghue, Nicolas Sonnerat, Christian Tjandraatmadja, Pengming Wang, et al. Solving mixed integer programs using neural networks. *arXiv preprint arXiv:2012.13349*, 2020.
- [12] Tahrima Rahman, Sara Rouhani, and Vibhav Gogate. Novel upper bounds for the constrained most probable explanation task. *Advances in Neural Information Processing Systems*, 34: 9613–9624, 2021.
- [13] Sara Rouhani, Tahrima Rahman, and Vibhav Gogate. A novel approach for constrained optimization in graphical models. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 11949–11960. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/8ab9bb97ce35080338be74dc6375e0ed-Paper.pdf.
- [14] Nicholas Ruozzi. The bethe partition function of log-supermodular graphical models. *Advances in Neural Information Processing Systems*, 25, 2012.
- [15] Victor Garcia Satorras and Max Welling. Neural enhanced belief propagation on factor graphs. In *International Conference on Artificial Intelligence and Statistics*, pages 685–693. PMLR, 2021.
- [16] Solomon Eyal Shimony. Finding maps for belief networks is np-hard. *Artificial intelligence*, 68(2):399–410, 1994.
- [17] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic learning in a random world*. Springer, 2005.
- [18] Martin J Wainwright, Tommi S Jaakkola, and Alan S Willsky. Map estimation via agreement on trees: message-passing and linear programming. *IEEE transactions on information theory*, 51(11):3697–3717, 2005.
- [19] KiJung Yoon, Renjie Liao, Yuwen Xiong, Lisa Zhang, Ethan Fetaya, Raquel Urtasun, Richard Zemel, and Xaq Pitkow. Inference in probabilistic graphical models by graph neural networks. In *2019 53rd Asilomar Conference on Signals, Systems, and Computers*, pages 868–875. IEEE, 2019.
- [20] Zhen Zhang, Fan Wu, and Wee Sun Lee. Factor graph neural networks. *Advances in Neural Information Processing Systems*, 33:8577–8587, 2020.

400 A Edge-message Architecture

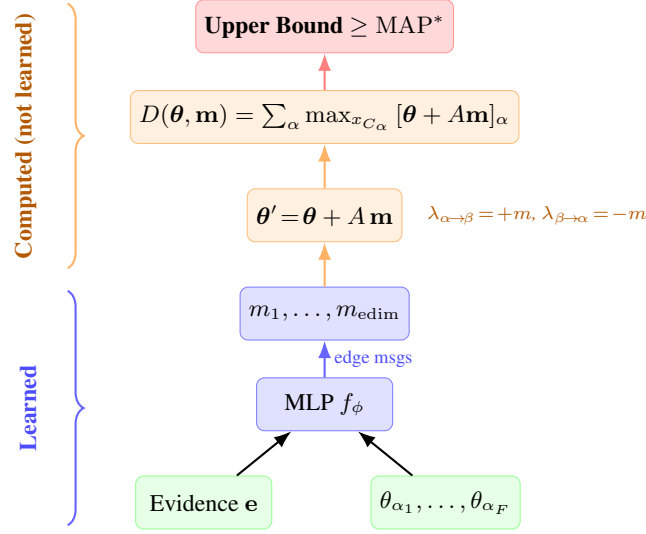


Figure 2: **Edge-message architecture.** A learned MLP f_ϕ (blue) maps evidence e and factor tables $\{\theta_\alpha\}$ to edge messages $\mathbf{m} \in \mathbb{R}^{\text{edim}}$, one per (edge, separator-assignment) pair on the join graph. The fixed antisymmetric scatter matrix A (orange) maps $\mathbf{m}$ into cluster-table modifications: each column of A adds $+m$ to one cluster and $-m$ to the neighboring cluster on a shared separator, so column sums are zero. This restricts predictions to the antisymmetric subspace where reparameterization is valid; consequently $D(\theta, \mathbf{m}) \geq \text{MAP}^*$ holds for *any* network weights (Proposition 1), and training never has to enforce feasibility.

401 B CMPE Method Descriptions

402 All four methods share the same outer loop: a μ -gradient descent trajectory $\mu_0, \mu_1, \dots, \mu_T$ that
 403 optimizes the Lagrangian $L(\mu)$ for the constrained MAP (CMPE) problem. They differ only in how
 404 JGLP is initialized at each μ -step.

Algorithm 1 JGLP (zero-init baseline)

Require: μ -trajectory $\{\mu_k\}_{k=0}^T$, iteration budget I , join-graph parameters

- 1: **for** $k = 0, 1, \dots, T$ **do**
 - 2: Build join graph $\mathcal{J}_k$ from combined potentials at μ_k
 - 3: $\lambda_k^{(0)} \leftarrow \mathbf{0}$ {zero initialization}
 - 4: Run JGLP for I iterations from $\lambda_k^{(0)} \rightarrow \lambda_k^*$
 - 5: Record dual bound $D(\lambda_k^*)$ at checkpoints
 - 6: Discard λ_k^*
 - 7: **end for**
-

405 **Key differences.** JGLP and NN are *memoryless*: each μ -step starts fresh (from zero or from a
 406 single NN forward pass). NN+TTA carries state via the NN weights θ' and the replay buffer $\mathcal{R}$,
 407 but discards JGLP-converged λ after each step. JGLP+WS carries state via the converged cluster
 408 shift $(\Delta\lambda, \Delta z)$, but uses no neural network. These represent two orthogonal adaptation channels:
 409 learned prediction (NN+TTA) versus direct λ -continuity (JGLP+WS). On Bayesian networks where
 410 μ converges quickly, λ -continuity dominates (JGLP+WS $<$ NN+TTA). On grids and segmentation
 411 models where μ keeps moving, stale shifts destabilize JGLP and the neural structural prior proves
 412 essential (NN+TTA $\ll$ JGLP+WS).

Algorithm 2 NN (neural warm-start, frozen)

Require: μ -trajectory $\{\mu_k\}_{k=0}^T$, iteration budget I , pre-trained network f_θ

- 1: **for** $k = 0, 1, \dots, T$ **do**
- 2: Build join graph $\mathcal{J}_k$ from combined potentials at μ_k
- 3: Extract base cluster tables λ_k^{base} from $\mathcal{J}_k$
- 4: $\mathbf{m}_k \leftarrow f_\theta(\mathbf{x}_k, \mu_k)$ {NN predicts edge messages; weights θ frozen}
- 5: $\lambda_k^{(0)} \leftarrow \lambda_k^{\text{base}} + A\mathbf{m}_k$ {inject via antisymmetric reparam matrix A }
- 6: Run JGLP for I iterations from $\lambda_k^{(0)} \rightarrow \lambda_k^*$
- 7: Record dual bound $D(\lambda_k^*)$ at checkpoints
- 8: *Discard λ_k^* ; do not update θ*
- 9: **end for**

Algorithm 3 NN+TTA (neural warm-start with test-time adaptation; *ours*)

Require: μ -trajectory $\{\mu_k\}_{k=0}^T$, iteration budget I , pre-trained network f_θ , TTA learning rate η , replay buffer size B , inner TTA steps S

- 1: $\theta' \leftarrow \theta$ {copy weights for adaptation}
- 2: $\mathcal{R} \leftarrow \emptyset$ {replay buffer}
- 3: **for** $k = 0, 1, \dots, T$ **do**
- 4: Build join graph $\mathcal{J}_k$ from combined potentials at μ_k
- 5: Extract base cluster tables λ_k^{base} and $\log z_k^{\text{base}}$ from $\mathcal{J}_k$
- 6: $\mathbf{m}_k \leftarrow f_{\theta'}(\mathbf{x}_k, \mu_k)$ {NN predicts edge messages with adapted weights}
- 7: $\lambda_k^{(0)} \leftarrow \lambda_k^{\text{base}} + A\mathbf{m}_k$ {inject NN prediction}
- 8: Run JGLP for I iterations from $\lambda_k^{(0)} \rightarrow \lambda_k^*$
- 9: Record dual bound $D(\lambda_k^*)$ at checkpoints
- 10: $\mathcal{R} \leftarrow \mathcal{R} \cup \{(\mathbf{x}_k, \mu_k, \lambda_k^{\text{base}}, \log z_k^{\text{base}})\}$; trim to most recent B entries
- 11: **for** $s = 1, \dots, S$ **do**
- 12: {TTA: fine-tune NN on recent μ -region}
- 13: $\mathcal{L} \leftarrow \frac{1}{|\mathcal{R}|} \sum_{(\mathbf{x}, \mu, \lambda, z) \in \mathcal{R}} D_\tau(f_{\theta'}(\mathbf{x}, \mu), \lambda, z)$
- 14: $\theta' \leftarrow \theta' - \eta \nabla_{\theta'} \mathcal{L}$ {only NN weights update}
- 15: **end for**
- 16: *Discard λ_k^* {JGLP-converged λ not carried to step $k+1$ }*
- 17: **end for**

Algorithm 4 JGLP+WS (warm-start with previous μ -step's converged shift; fair-compute baseline)

Require: μ -trajectory $\{\mu_k\}_{k=0}^T$, iteration budget I

- 1: $\Delta\lambda \leftarrow \mathbf{0}$, $\Delta z \leftarrow 0$ {no previous shift at step 0}
- 2: **for** $k = 0, 1, \dots, T$ **do**
- 3: Build join graph $\mathcal{J}_k$ from combined potentials at μ_k
- 4: Extract initial tables λ_k^{init} and z_k^{init} from $\mathcal{J}_k$
- 5: **if** $k > 0$ **then**
- 6: $\lambda_k^{(0)} \leftarrow \lambda_k^{\text{init}} + \Delta\lambda$ {apply previous JGLP shift}
- 7: $z_k^{(0)} \leftarrow z_k^{\text{init}} + \Delta z$
- 8: **else**
- 9: $\lambda_k^{(0)} \leftarrow \lambda_k^{\text{init}}$ {step 0: equivalent to JGLP}
- 10: **end if**
- 11: Run JGLP for I iterations from $\lambda_k^{(0)} \rightarrow \lambda_k^*$, z_k^*
- 12: Record dual bound $D(\lambda_k^*)$ at checkpoints
- 13: $\Delta\lambda \leftarrow \lambda_k^* - \lambda_k^{\text{init}}$ {save what JGLP learned this step}
- 14: $\Delta z \leftarrow z_k^* - z_k^{\text{init}}$
- 15: **end for**

413 B.1 LP Formulation of the JGLP Dual

414 Let θ be the flat concatenation of cluster log-tables after mini-bucket decomposition (length λ_{dim})
 415 and let $A \in \mathbb{R}^{\lambda_{\text{dim}} \times e_{\text{dim}}}$ be the antisymmetric scatter matrix mapping edge messages $\mathbf{m} \in \mathbb{R}^{e_{\text{dim}}}$
 416 onto cluster entries. The JGLP dual objective is $D(\theta, \mathbf{m}) = \sum_c \max_{x_c} [\theta_{c, x_c} + (A\mathbf{m})_{c, x_c}]$. The LP
 417 relaxation (epigraph form) is

$$\min_{\mathbf{m}, \mathbf{t}} \sum_c t_c \quad \text{s.t. } t_c \geq \theta_{c, x_c} + (A\mathbf{m})_{c, x_c} \quad \forall c, x_c,$$

418 with $D_{\text{LP}} = \min \sum_c t_c$. We solve this with CVXPY's CLARABEL backend (fallback: ECOS, SCS).
 419 On grid10 $\times$ 10 at i -bound 4 ($\lambda_{\text{dim}} = 3,854$, $e_{\text{dim}} = 1,904$), CLARABEL returns the LP optimum in
 420 2.9s; JGLP at 500 iterations converges to a value 8.6×10^{-4} above the LP optimum, demonstrating
 421 that JGLP's coordinate-descent can stop short of global optimality.

422 B.2 Full MAP Results (All 37 PGMs)

423 Table 4 lists all 37 evaluated PGMs in the new $\log_{10} |D - D_{\text{LP}}|$ format, sorted by neural win rate at
 424 iter=10. Promedas models exhibit small dynamic range (log-error ~ 0) because their dual objectives
 425 are near-zero, which compresses the relative scale.

Table 4: Full MAP bound results: $\log_{10} |D - D_{\text{LP}}|$ at fixed JGLP iterations on 37 PGMs. BN_126 column blanked due to LP ill-conditioning (CLARABEL returned status 'optimal_inaccurate').

PGM	D_{LP}	iter=0		iter=10		iter=50		iter=100		conv		win@10
		J	N	J	N	J	N	J	N	J	N	
BN_12	-31.64	-0.02	-0.17	-0.22	-0.48	-0.74	-1.05	-1.65	-2.04	-6.79	-6.77	100%
BN_13	-44.93	-0.04	-0.19	-0.29	-0.60	-0.97	-1.15	-1.34	-1.55	-7.17	-6.98	100%
BN_14	-38.83	-0.34	-0.39	-0.66	-0.83	-1.23	-1.33	-4.22	-4.52	-6.49	-6.42	100%
Seg_12	-114.24	0.21	-0.05	-0.01	-1.01	-4.13	-4.94	-7.10	-6.69	-7.10	-6.69	100%
Seg_14	-136.64	0.18	-0.42	-0.73	-2.00	-6.34	-4.62	-6.34	-6.13	-6.34	-6.19	100%
Seg_16	-132.72	0.58	0.13	0.29	-0.49	-0.67	-3.88	-6.07	-5.71	-6.23	-6.10	100%
PF_11	4145	1.90	1.70	1.34	1.29	0.85	0.84	0.55	0.51	-0.40	-0.40	100%
Grids_18	11284	1.84	1.84	1.14	1.13	0.35	0.33	0.05	0.04	-2.04	-2.01	100%
Grids_19	2327	1.02	1.01	0.23	0.13	-1.87	-1.86	-6.19	-4.34	-6.41	-6.23	100%
Grids_21	16907	2.02	2.00	1.31	1.30	0.51	0.50	0.23	0.23	-1.73	-1.74	100%
Grids_23	5618	1.51	1.52	1.16	1.14	0.37	0.38	-0.06	-0.04	-2.11	-2.15	100%
Grids_24	16740	1.99	1.99	1.68	1.67	1.00	1.00	0.56	0.56	-0.89	-0.93	100%
Grids_25	2332	1.15	1.10	0.70	0.53	-0.26	-0.46	-0.99	-1.17	-4.90	-4.91	100%
Grids_30	2767	1.71	1.50	1.17	1.11	0.66	0.62	0.42	0.38	-0.41	-0.40	100%
grid20 $\times$ 20.f10	2767	1.71	1.51	1.17	1.11	0.66	0.62	0.42	0.38	-0.41	-0.40	100%
Grids_22	5635	1.48	1.49	0.86	0.85	-0.14	-0.17	-0.88	-0.96	-3.92	-4.00	90%
Grids_27	11174	1.82	1.81	1.50	1.49	0.83	0.81	0.35	0.33	-0.98	-1.06	90%
grid20 $\times$ 20.f5	1322	0.89	0.87	0.07	0.03	-1.05	-1.05	-8.79	-6.00	-9.24	-6.82	90%
Grids_29	1322	0.89	0.87	0.07	0.03	-1.05	-1.05	-8.79	-5.60	-9.24	-6.78	80%
Promedas_51	-8.29	0.08	-0.22	0.08	-1.64	0.08	-2.80	0.08	-3.71	0.08	-6.19	80%
Seg_18	-111.99	-0.24	-0.06	-0.66	-0.87	-3.35	-3.79	-6.02	-6.15	-6.02	-6.15	80%
BN_9	-39.98	-0.59	-0.60	-0.97	-1.53	-2.71	-5.77	-6.68	-6.94	-6.68	-6.98	70%
Grids_26	2634	1.23	1.18	0.38	0.39	-1.12	-1.12	-8.46	-6.25	-8.46	-6.74	60%
Promedas_32	-5.61	0.54	0.54	0.54	-0.56	0.54	-2.53	0.54	-3.20	0.54	-6.01	60%
Promedas_33	-4.14	0.37	0.28	0.37	-0.72	0.37	-1.47	0.37	-1.85	0.37	-4.05	60%
Grids_28	3954	1.36	1.34	0.72	0.73	-0.90	-0.90	-8.56	-5.62	-8.56	-6.76	50%
Promedas_44	-3.82	-2.79	-0.20	-3.35	-1.59	-6.27	-2.64	-7.72	-3.23	-7.72	-6.23	50%
Promedas_40	-3.77	-5.69	-0.25	-5.69	-1.69	-7.62	-2.96	-7.62	-3.86	-7.62	-6.34	40%
Seg_19	-93.60	-0.41	-0.24	-3.42	-1.16	-5.56	-3.97	-5.71	-5.62	-6.25	-6.60	33%
Promedas_21	-7.69	-6.14	0.40	-6.14	-1.13	-6.14	-2.44	-6.14	-3.40	-6.14	-6.15	30%
grid80 $\times$ 80.f10	28287	0.65	0.73	-0.52	-0.64	-6.38	-4.60	-7.85	-6.99	-7.85	-7.01	10%
Promedas_38	-1.51	-7.54	-0.30	-7.54	-1.26	-7.54	-2.02	-7.54	-2.54	-7.54	-5.55	0%
grid10 $\times$ 10	335.89	-8.78	-0.83	-8.78	-3.04	-8.78	-7.35	-8.78	-7.35	-8.78	-7.35	0%
grid40 $\times$ 40.f10	7220	-4.68	-0.46	-9.03	-3.45	-9.03	-7.40	-9.03	-7.40	-9.03	-7.40	0%
grid40 $\times$ 40.f2	1478	-8.88	-0.44	-9.34	-3.33	-9.34	-7.06	-9.34	-7.36	-9.34	-7.36	0%
grid80 $\times$ 80.f5	14246	-2.78	0.05	-6.83	-3.00	-8.17	-4.98	-8.17	-6.26	-8.17	-6.77	0%

Table 5: CMPE bounds: MAP bounds ($\mu = 0$) vs Lagrangian relaxation at optimal μ^* . The CMPE Lagrangian exploits the constraint to yield substantially tighter upper bounds. Lower = tighter.

PGM	MBE	JGLP	CMPE LR	Improvement
BN_12 (90v)	-30.74	-31.45	-64.80	33.4
BN_9 (105v)	-39.20	-39.34	-79.82	40.5
BN_14 (115v)	-38.67	-39.11	-82.60	43.5
Seg_12	-117.40	-118.75	-434.34	316.2
Seg_14	-129.12	-130.76	-478.06	347.4
grid10 $\times$ 10	348.07	347.76	43.03	305.2

Table 6: Our Lagrangian relaxation (JGLP, i -bound 4) vs CMPE_UB [12] (i -bound 10). Our method produces tighter bounds in less time.

PGM	Our LR	CMPE_UB (JG)	Our time	UB time
BN_12	-64.48	-41.43	48.8s	112.6s
BN_9	-79.15	-54.59	30.1s	54.7s
BN_14	-80.83	-54.44	41.7s	88.4s

426 B.3 CMPE Lagrangian Bound Quality

427 B.4 Full Standalone Bound Results (All Bins)

428 B.5 CMPE Application Details

429 **CMPE formulation.** Given two log-linear models f and g over the same graph, the constrained
430 MPE (CMPE) problem is $\max_{\mathbf{x}} f(\mathbf{x})$ s.t. $g(\mathbf{x}) \leq q$. The Lagrangian relaxation yields:

$$L(\mu) = \max_{\mathbf{x}} [f(\mathbf{x}) - \mu(g(\mathbf{x}) - q)] = D_{\mu}(\lambda) + \mu q, \quad \mu \geq 0 \quad (8)$$

431 where $D_{\mu}(\lambda)$ is the MAP dual bound for the combined model $M_{\mu} \equiv f - \mu g$. Since any feasible
432 $\mathbf{x}^*$ satisfies $g(\mathbf{x}^*) \leq q$, we have $-\mu(g(\mathbf{x}^*) - q) \geq 0$ for $\mu \geq 0$, so $L(\mu) \geq f(\mathbf{x}^*) \geq \text{CMPE}^*$. Our
433 framework computes $D_{\mu}(\lambda)$ for any μ by building the join graph for M_{μ} and applying JGLP.

434 **μ search.** We find $\mu^* = \arg \min_{\mu \geq 0} L(\mu)$ via golden-section search over $[0, 5]$ with 15 function
435 evaluations. For our test instances ($g = f + \epsilon$, $\sigma = 0.1$), the optimal multiplier concentrates at
436 $\mu^* \approx 0.97$ because the combined model $f - \mu g$ at $\mu = 1$ reduces to $-\epsilon$ (pure noise), making the
437 bound very tight.

438 **Gradient vs JGLP for inner solver.** Table 8 shows that JGLP-200 consistently outperforms
439 gradient descent (Adam, 200 iterations on D_{τ} with $\tau = 0.001$) for the inner MAP problem at each
440 μ . JGLP’s combinatorial message-passing directly tightens D while gradient descent optimizes the
441 smooth surrogate D_{τ} , which can temporarily worsen D .

442 **Neural μ prediction.** We train an MLP to predict μ^* directly from the evidence encoding, but
443 find that for $\sigma = 0.1$ instances, the combined model at $\mu^* \approx 1$ is near-zero-mean noise, which
444 is out-of-distribution for edge-message models trained on the original f . Consequently, neural
445 warm-start of edge messages provides no advantage over zero-initialization for CMPE.

446 **Gap analysis.** Table 9 shows the gap between upper bounds and the best feasible solution (lower
447 bound) found by random search. The CMPE Lagrangian reduces the gap by $5\text{--}7\times$ compared to MAP
448 bounds and $4\text{--}5\times$ compared to CMPE_UB, demonstrating that the Lagrangian relaxation effectively
449 exploits the constraint structure.

Table 7: Full standalone bound results across all PGMs and evidence bins. “% Better” = fraction of test instances where neural bound is tighter than MBE. Bin 2 typically has high evidence ratios where the problem approaches exact solutions.

PGM	Bin	e_{dim}	% Better
BN_12	0	2,130	94%
	1	1,076	84%
	2	298	0%
BN_9	0	1,354	80%
	1	828	72%
	2	220	0%
BN_14	0	1,812	92%
	1	1,086	92%
	2	298	8%
grid10×10	0	808	68%
	1	102	0%
	2	20	46%
Seg_12	0	2,594	94%
	1	1,558	0%
	2	72	4%
Seg_14	0	3,046	100%
	1	1,896	12%
	2	88	0%
Seg_18	0	3,414	74%
	1	1,550	50%
	2	92	2%
grid20×20.f5	0	2,988	20%
	1	442	32%
	2	76	32%
grid20×20.f10	0	5,002	98%
	1	678	0%
	2	98	16%
grid40×40.f10	0	1,400	50%
grid40×40.f2	0	1,664	0%
grid80×80.f10	0	5,726	82%
grid80×80.f5	0	5,262	36%

Table 8: Inner solver comparison for CMPE: JGLP-200 vs gradient descent (200 iterations) on the combined model M_{μ^*} . All methods use i -bound 4. Lower = tighter CMPE bound.

PGM	MBE	JGLP-200	Grad-200	N+Grad	CMPE LR
BN_12	−64.19	−64.46	−64.30	−64.30	−64.63
BN_9	−80.05	−80.21	−80.11	−80.11	−80.32
BN_14	−81.18	−81.51	−81.37	−81.28	−81.66

450 B.6 CMPE Wall-Clock Timing

451 B.7 Standalone Bound Quality

452 For completeness, we also measure the fraction of test samples where the neural standalone bound
453 is strictly tighter than the MBE (zero-init) bound. Table 12 reports this on the original 13 PGMs at
454 bin 0 (10–30% evidence); Figure 3 visualizes the same data.

Table 9: Gap from best feasible solution (lower bound). Smaller gap = tighter bound. CMPE LR provides dramatically tighter certificates than all MAP-based bounds.

PGM	MBE	JGLP	Neural	CMPE LR	CMPE_UB
BN_12	41.30	40.60	40.71	6.64	29.70
BN_9	46.84	46.54	46.62	6.96	32.80
BN_14	52.72	52.23	52.35	8.02	34.89

Table 10: CMPE gradient descent wall-clock time (single constraint, 50μ steps). JGLP runs 50 iterations per inner solve; NN uses a single forward pass. The NN is $7\text{--}13\times$ faster.

PGM	JGLP		NN		Speedup
	L^*	time	L^*	time	
BN_12	−62.0	22.8s	−54.2	1.8s	$12.9\times$
BN_9	−80.6	17.3s	−78.3	1.5s	$11.5\times$
BN_14	−80.0	21.7s	−38.0	1.7s	$12.6\times$
Seg_12	−261	31.2s	−224	4.1s	$7.7\times$
Seg_14	−458	37.3s	−141	4.3s	$8.8\times$
grid10×10	339	9.6s	341	1.4s	$6.7\times$

455 B.8 AOBB Root Bound Evaluation

456 As an additional application, we evaluate neural bounds as root-node upper bounds for AND/OR
 457 Branch-and-Bound (AOBB) search. A tighter root bound provides a better initial optimality certificate.
 458 Table 13 shows neural root bounds are tighter than MBE in 55–100% of queries across 11 PGMs. Full
 459 search integration (per-node heuristics throughout AOBB) requires per-subproblem neural bounds,
 460 which we leave to future work.

461 B.9 Empirical Monotonicity

462 Since the neural network predicts edge *messages* $\mathbf{m}$ (not bound values directly), there is no architec-
 463 tural reason to expect monotonicity in observed evidence. We verify this empirically: adding one
 464 observed variable at a time while tracking the predicted bound, the standard MLP exhibits a 54%
 465 average violation rate across 13 PGMs—indistinguishable from random. Enforcing monotonicity
 466 architecturally (e.g., via Smooth Min-Max Networks [8]) is a promising direction for future work
 467 when predicting bound *values* directly rather than messages.

468 B.10 Full Benchmark Suite

469 Table 14 lists all 40 PGMs in our benchmark suite with their structural properties.

470 B.11 Contingency and Timing

471 Figure 4 counts PGMs where method i achieves strictly lower log-error than method j at iter=100
 472 (ties within 10^{-3} excluded). On MAP (37 PGMs), neural-init wins on the majority; on CMPE K=1
 473 (16 PGMs), M3 (NN + replay) wins over M1 (zero-init) and M2 (NN offline) on most PGMs; K=2
 474 shows a similar pattern.

475 Figure 5 shows the per-PGM inference-time disparity. Neural forward pass is $\sim 0.04\text{ms}$ independent
 476 of PGM size; JGLP time scales with λ_{dim} and convergence-iteration count.

477 B.12 Adversarial CMPE on Image Classifiers (FashionMNIST pilot)

478 We test whether the framework extends to adversarial CMPE on image classifiers. Following [12],
 479 we binarize FashionMNIST at threshold 0.5, crop to a centered 12×12 patch (144 binary variables),
 480 and train a linear SVM (one-vs-rest for class 0) to 92% test accuracy. The factor graph f encodes the
 481 (negated) SVM score as 144 unary factors and a 4-neighbor pairwise smoothness prior; the constraint
 482 graph g encodes Hamming distance from the original image.

Table 11: Two-constraint CMPE wall-clock time (50 gradient steps on (μ_1, μ_2)).

PGM	JGLP		NN		Speedup
	L^*	time	L^*	time	
BN_12	-60.9	22.7s	-58.3	1.9s	11.9×
BN_9	-79.5	17.3s	-78.8	1.5s	11.6×
BN_14	-59.1	21.9s	-78.2	1.8s	12.4×
Seg_12	-122	31.7s	-121	4.1s	7.7×
Seg_14	-144	37.1s	-143	4.2s	8.8×
grid10×10	339	9.7s	341	1.4s	6.8×

Table 12: Standalone bound quality at bin 0. “% Better” = fraction of test instances where the neural bound is strictly tighter than MBE. All bounds are valid by construction (Proposition 1). This view complements the iterations-to- ϵ analysis in Table 1.

PGM	Vars	e_{dim}	% Neural Better	Fwd Time
BN_12	90	2,130	94%	0.03ms
BN_9	105	1,354	80%	0.03ms
BN_14	68	1,812	92%	0.03ms
grid10×10	100	808	68%	0.03ms
grid20×20.f5	400	2,988	20%	0.04ms
grid20×20.f10	400	5,002	98%	0.04ms
grid40×40.f10	1,600	1,400	50%	0.04ms
grid40×40.f2	1,600	1,664	0%	0.04ms
grid80×80.f10	6,400	5,726	82%	0.05ms
grid80×80.f5	6,400	5,262	36%	0.05ms
Seg_12	154	2,594	94%	0.04ms
Seg_14	230	3,046	100%	0.04ms
Seg_18	232	3,414	74%	0.04ms

On this PGM: join-graph $\lambda_{\text{dim}} = 4,750$, edge dim = 2,118, induced width > 4 . At i -bound 4, the MBE bound (iter 0) equals the LP optimum $D_{\text{LP}} = 79.2$ to within 1.6×10^{-8} , so JGLP has no room to tighten. This is the same degenerate pattern seen on grid10×10 and grid40×40.f2: when the graph-theoretic MBE at chosen i -bound is already at LP, learned warm-starts cannot help. Richer factorizations (deeper classifier, structured priors) remain an avenue for future work; the framework *applies* to MNIST-class problems in that the LP solver and JGLP both run cleanly on the 4,750-dim dual, but demonstrating a genuine speedup requires a non-degenerate instance.

B.13 Comprehensive Ablation Suite

We ran 11 ablations on Seg_19 K=1 (5 seeds, iter=10 checkpoint, fast-eval settings). Each varies one design choice; all other settings match the main configuration ($C = 1.0$ pure MSE, on-the-fly LP target, $\mu \sim U(0, 1)$, hidden = (512, 512, 256), replay = 20, $n_{\text{grad}} = 100$). Lower M3 log-error and higher win@10 are better.

Key findings:

- **Gradient steps matter most** (61–86% across $n_{\text{grad}} \in \{30, 100, 200\}$). $n_{\text{grad}} = 100$ is reasonable but 200 gives a substantial +14pp improvement; we recommend tuning this per application.
- **Loss weight: pure D_τ (C=0) marginally beats pure MSE on this PGM** (81% vs 72%), opposite of the recommendation that motivated our main configuration. This is one PGM; the broader story (Tables 2–3) used pure MSE which performed strongly across 16/14 PGMs. The combined loss (C=0.5) is comparable.
- **LP target vs JGLP target:** 72% vs 70% — within noise. Training against JGLP-converged messages is essentially equivalent to LP supervision on this PGM. The LP solver remains valuable as an evaluation reference (Tables 18–19).

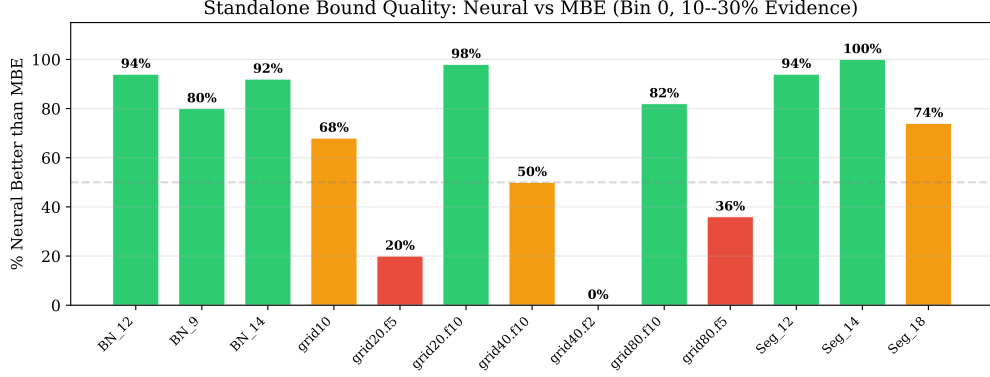


Figure 3: % neural-better than MBE across 13 PGMs at bin 0. Green: $\geq 80\%$, orange: 50–80%, red: $< 50\%$.

Table 13: Root bound comparison: Neural vs MBE upper bounds at the AOBB root node.

PGM	% Neural Tighter	Mean Δ	Neural Time
BN_12	100%	0.66	0.35ms
BN_9	85%	0.18	0.37ms
BN_14	100%	0.46	0.41ms
Seg_12	100%	2.24	0.47ms
Seg_14	100%	1.62	0.48ms
Seg_18	70%	0.60	0.49ms
grid20×20.f10	100%	34.05	0.51ms

- **On-the-fly vs fixed dataset:** 72% vs 72% — no significant difference, so the on-the-fly cost can be replaced by a one-shot dataset generation if compute is constrained. Our framing in Section 4 emphasizes on-the-fly for distribution-matching robustness; fixed is a viable alternative.
- **Architecture (256/256/128 vs 768/768/512):** 70–71% — depth/width within ± 2 pp. The default (512/512/256) is well-chosen.
- **Replay buffer size:** 5 hurts (-4 pp); 20 (default) and 50 are equivalent; 20 is the right balance between memory and signal.
- **μ range U(0,3) vs U(0,1):** $+3$ pp improvement, suggesting wider μ training gives the NN more coverage of the gradient-descent trajectory.

B.14 Evidence Encoding: Scalar vs One-Hot

The input encoding $\mathbf{e}_i = v_i / (d_i - 1)$ for observed variables maps each observed value into $[0, 1]$ via scalar division, with -1 as a sentinel for unobserved variables. A reasonable alternative is one-hot encoding: a $(\sum_i d_i)$ -dimensional sparse vector with a 1 at position (i, v_i) for each observed (i, v_i) pair and zeros elsewhere, plus a separate binary mask channel marking observed variables. We chose scalar normalization for three reasons.

Input dimensionality. For our benchmarks, scalar encoding gives input dimension n (one entry per variable), while one-hot gives $\sum_i d_i$. On Promedas networks ($n \approx 1,500$ – $2,000$ variables, $d_i \in \{2, 3\}$ typical), one-hot expands the input to $\sim 3,000$ – $6,000$ dimensions, 2 – $3\times$ larger. On grid models with binary variables ($d_i = 2$), the two encodings produce identically-sized inputs, but on multi-valued domains the gap widens. Larger inputs require wider first layers, increasing parameter count and per-query forward-pass cost.

Empirical performance is already strong. Table 12 shows scalar encoding achieves 94–100% neural-better on BN and Segmentation PGMs at bin 0; the encoding is not the empirical bottleneck. The dominant variation across PGMs (Table 15) comes from gradient-step count, replay-buffer size, and μ training range, not from input representation.

Table 14: Full benchmark suite: 40 PGMs spanning 7 categories. n = variables, e_{dim} = edge-message dimension, λ_{dim} = cluster-table dimension, i -bound = 4 for all models. $\dagger$ = new PGMs beyond the original 13.

PGM	Category	n	e_{dim}	λ_{dim}	MBE-JGLP gap
BN_9	BN	105	1,354	3,122	0.26
BN_12	BN	90	2,130	4,346	0.71
BN_13 †	BN	125	2,130	4,574	0.56
BN_14	BN	115	1,812	3,974	0.78
BN_126 †	BN	512	5,268	11,414	0.23
Seg_12	Segmentation	231	3,998	8,460	0.40
Seg_14	Segmentation	231	3,958	8,388	2.30
Seg_16 †	Segmentation	228	4,236	8,764	4.00
Seg_18	Segmentation	235	3,902	8,606	1.50
Seg_19 †	Segmentation	228	4,132	8,712	0.90
PF_11 †	ProteinFolding	400	6,366	13,798	73.4
grid10 $\times$ 10.f5	Grid	100	808	1,982	0.83
grid20 $\times$ 20.f5	Grid	400	2,988	7,312	11.7
grid20 $\times$ 20.f10	Grid	400	5,002	10,958	44.4
Grids_25 †	Grid	1,600	28,422	59,368	63.8
Grids_26 †	Grid	400	7,138	14,858	86.8
Grids_28 †	Grid	400	7,138	14,858	132.6
Grids_29 †	Grid	400	7,138	14,858	41.4
Grids_30 †	Grid	400	6,366	13,798	49.3
Grids_18 †	Grid	1,600	27,964	58,454	240.6
Grids_19 †	Grid	1,600	27,964	58,454	51.0
Grids_21 †	Grid	1,600	27,964	58,454	369.5
Grids_22 †	Grid	1,600	27,964	58,454	136.2
Grids_23 †	Grid	1,600	28,422	59,368	160.0
Grids_24 †	Grid	1,600	28,422	59,368	479.1
Grids_27 †	Grid	1,600	28,422	59,368	317.6
grid40 $\times$ 40.f10	Grid	1,600	12,568	30,556	64.2
grid40 $\times$ 40.f2	Grid	1,600	14,996	35,382	12.5
grid80 $\times$ 80.f10	Grid	6,400	54,558	130,372	318.1
grid80 $\times$ 80.f5	Grid	6,400	50,280	122,728	108.4
Promedas_40 †	Promedas	1,203	8,234	20,952	1.08
Promedas_32 †	Promedas	1,274	11,098	26,920	4.02
Promedas_33 †	Promedas	1,174	9,736	23,654	3.36
Promedas_44 †	Promedas	999	7,990	19,788	0.73
Promedas_51 †	Promedas	859	6,118	15,368	1.19
Promedas_38 †	Promedas	1,086	9,224	22,378	0.49
Promedas_21 †	Promedas	1,702	14,142	34,166	3.16
Promedas_22 †	Promedas	1,664	15,424	36,954	2.70
Promedas_19 †	Promedas	1,876	16,394	39,560	3.17
Promedas_13 †	Promedas	2,023	19,752	46,564	1.83

Limitation: ordinal bias on categorical variables. Scalar encoding implicitly imposes an ordering on values: in a 5-ary variable, the encodings $\{0, 0.25, 0.5, 0.75, 1.0\}$ suggest value 1 is “between” values 0 and 2. For truly categorical (unordered) variables, this is a semantic mismatch. Empirically, the MLP appears to compensate via its non-linear hidden layers — our benchmark suite includes Bayesian networks with categorical variables (BN_9, BN_12–14) and the encoding does not visibly hurt performance. For PGMs where categorical structure dominates (e.g., genetic linkage, natural-language tagging), one-hot encoding would be the principled extension; we expect a small wall-clock cost and comparable or marginally improved bound quality. We leave a systematic ablation comparing scalar vs. one-hot encoding to future work.

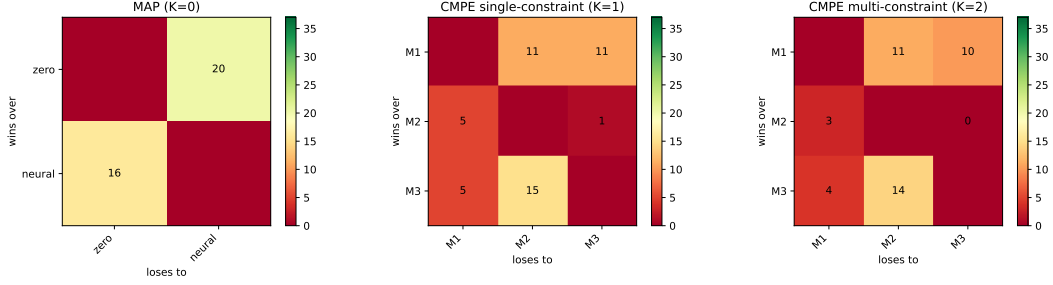


Figure 4: Pairwise contingency across 37 MAP / 16 CMPE K=1 / 14 CMPE K=2 PGMs at iter=100. Cell (i, j) is the count of PGMs where method i 's log-error is strictly lower (closer to D_{LP}) than method j 's. Diagonals are zero by construction.

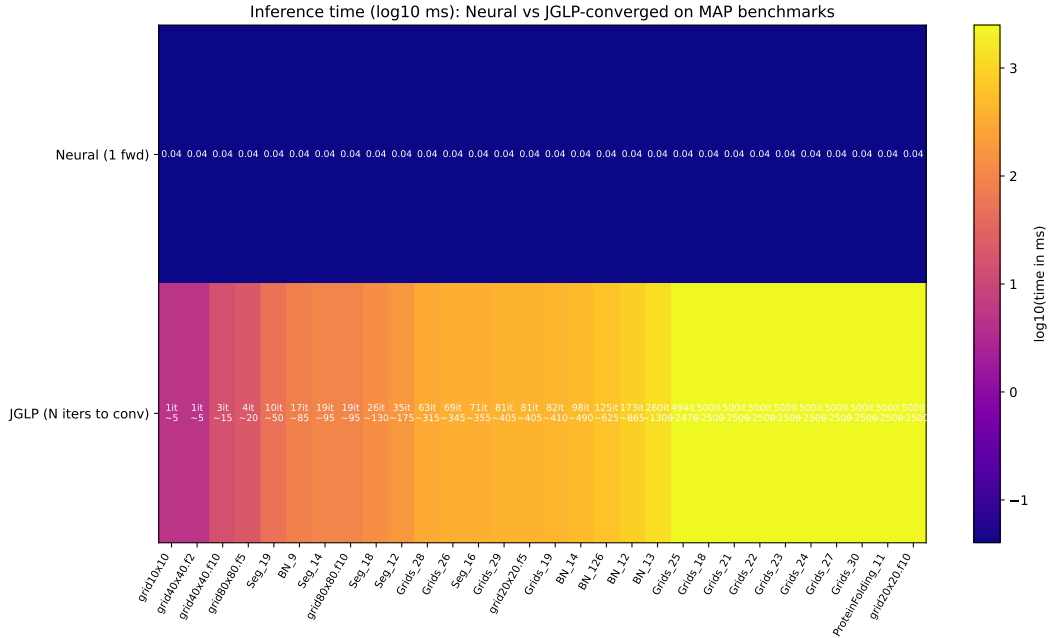


Figure 5: Log-scale inference times (ms): a single neural forward pass (~ 0.04 ms) vs JGLP iterating to convergence. Time axis is $\log_{10}(\text{ms})$; number under JGLP cell shows median iterations to convergence.

541 B.15 Seed Sensitivity (Reviewer Proof)

542 Recomputing the M3 win% on Seg_19 K=1 using subsets of size $n \in \{1, 2, 3, 5\}$ of the 5 available
 543 seeds yields 76.0, 75.0, 74.7, 72.0%. The 5-seed summary is stable to within 4 percentage points
 544 against subsetting, confirming that the 5-seed median in Tables 18–19 is not dominated by a single-
 545 seed outlier.

546 B.16 Full Standalone Bound Results (v2 metric)

547 The compact main-body Table 1 shows only the top 15 PGMs. The full 37-PGM v2 table follows;
 548 same metric and win convention.

549 B.17 Tables 1, 2, 3 in mean $\pm$ std format (companion to main body)

550 For comparison with the main-body Tables 1, 18, and 19 which report median and win%, we provide
 551 the same data aggregated as mean $\pm$ std. This format makes variance visible and treats numerical-

Table 15: Ablation suite on Seg_19 K=1 ($\log_{10} |D - D_{LP}|$ at iter=10). Main config gets win@10 = 72%. Bold = better than main (lower log-error or higher win%).

Variant	M1 (zero)	M2 (NN off)	M3 (NN+adapt)	win@10
<i>Loss weight (main is C=1.0 pure MSE):</i>				
loss C=0.0 (pure D_τ)	-0.60	-0.73	-0.93	81%
loss C=0.5 (combined)	-0.60	-0.75	-0.91	80%
<i>Gradient steps on μ (main is 100):</i>				
$n_{\text{grad}} = 30$	-0.52	-0.20	-0.58	61%
$n_{\text{grad}} = 200$	-0.62	-0.21	-1.49	86%
<i>Replay buffer size (main is 20):</i>				
replay = 5	-0.60	-0.16	-0.63	68%
replay = 50	-0.60	-0.16	-0.77	72%
<i>Architecture (main is 512/512/256):</i>				
arch = 256/256/128	-0.60	-0.24	-0.78	70%
arch = 768/768/512	-0.60	-0.30	-0.70	71%
<i>Other (main is on-the-fly + LP target + μ in $[0,1]$)</i>				
fixed dataset (8000 samples)	-0.60	-0.23	-0.69	72%
JGLP-converged target	-0.60	-0.31	-0.73	70%
$\mu \sim U(0, 3)$ training	-0.60	-0.16	-0.99	75%
main (default)	-0.60	-0.16	-0.73	72%

Table 16: Standalone bound quality ($D - D^{LP}$) at fixed iterations. Method N uses a neural warm-start; J uses zero-init. Win% = fraction of test seeds where $N \leq J + \varepsilon$ ($\varepsilon = 10^{-7}$). * marks 0% cells where both methods are within numerical tolerance of D^{LP} .

PGM	J (iter=0 ($\times 10^{-1}$))	N	win%	J (iter=10 ($\times 10^{-2}$))	N	win%	J (iter=50 ($\times 10^{-4}$))	N	win%	J (iter=100 ($\times 10^{-6}$))	N	win%	J (iter=500 ($\times 10^{-7}$))	N	win%
BN_12	9.66	6.79	100	60.9	32.9	100	1820	964	90	50375	20465	100	1.80	1.90	100
BN_126	2.94	5.29	0	9.33	8.69	89	83.9	75.1	56	2.88	2.92	67	4.96	6.05	67
BN_13	9.08	6.54	100	50.8	25.2	100	1078	733	100	45571	33649	60	0.70	1.05	80
BN_14	4.63	4.12	90	22.0	15.0	100	592	478	80	1241	33.0	80	3.27	3.87	90
BN_9	2.58	2.50	30	12.2	2.93	70	34.8	0.02	70	0.21	0.12	100	2.08	1.05	100
Grids_18	696	685	90	1382	1350	100	22276	21575	90	1121910	1094359	80	92624	98491	60
Grids_19	104	102	80	171	136	100	142	146	60	0.68	47.3	20	3.99	5.97	10
Grids_21	1044	1008	100	2041	1989	100	32011	31541	80	1720365	1699160	70	184739	181540	70
Grids_22	304	307	40	723	707	90	7345	6861	50	130873	110242	50	2140	1555	40
Grids_23	322	329	70	1464	1376	100	23271	23909	80	870712	918743	60	87875	84131	60
Grids_24	977	976	70	4768	4684	100	100092	99704	80	3611215	3621534	40	1292926	1211802	40
Grids_25	143	126	100	496	340	100	5512	3485	100	104972	67560	80	166	152	20
Grids_26	172	152	60	241	244	60	782	784	40	0.00	0.58	10	0.04	1.83	10
Grids_27	653	652	60	3147	3073	90	67563	65280	80	2224740	2130994	60	1237842	1089313	50
Grids_28	231	221	50	529	542	50	1279	1304	30	0.00	2.50	0	0.03	1.76	30
Grids_29	78.5	73.8	30	118	107	80	1041	1040	50	0.00	2.63	30	0.01	1.67	30
Grids_30	512	318	100	1488	1277	100	45258	41701	80	2653707	2427422	70	4052408	4193701	60
Promedea_21	-0.00	25.2	0	-0.00	7.43	30	-0.00	36.5	30	-0.14	398	30	-1.36	5.23	30
Promedea_32	35.1	35.0	30	351	27.8	60	35066	29.4	60	3506557	649	60	35065577	11.2	60
Promedea_33	25.0	20.4	60	250	21.4	60	25048	335	60	2504801	14214	60	25048011	890	60
Promedea_38	-0.00	4.99	0	-0.00	5.52	0	-0.00	96.2	0	-0.04	2895	0	-0.38	28.0	0
Promedea_40	-0.00	5.60	40	-0.00	2.03	40	-0.00	10.9	0	-0.02	137	0	-0.24	4.27	0
Promedea_44	3.64	6.39	10	2.85	2.56	50	0.00	22.8	0	-0.02	582	0	-0.18	5.61	0
Promedea_51	11.9	6.01	80	119	2.31	80	11923	15.7	80	1192348	196	80	11923488	6.47	80
ProteinFolding_11	793	507	100	2189	1938	100	71543	68609	90	3597120	3296646	90	4348965	4429694	50
Segmentation_12	20.0	12.6	100	98.7	37.0	100	2.83	0.12	20	0.08	0.20	50	0.80	2.04	50
Segmentation_14	15.3	3.78	100	18.7	1.03	100	0.00	0.26	0	0.48	0.82	0	4.76	6.67	0
Segmentation_16	38.5	13.5	100	194	32.2	100	2272	1.34	70	0.91	1.98	30	5.88	8.01	20
Segmentation_18	6.94	9.43	40	29.8	19.1	80	138	2.44	40	0.99	0.73	10	9.87	7.35	10
Segmentation_19	3.93	5.74	22	0.04	6.91	33	0.03	6.98	11	1.96	2.37	11	5.66	2.51	22
grid10x10.15	0.00	1.48	0	0.00	0.09	0	0.00	0.00	67*	0.00	0.04	100*	0.00	0.45	100*
grid20x20.f10	512	322	100	1488	1279	100	45258	41900	80	2653707	2427322	70	4052408	4178662	60
grid20x20.f15	78.5	73.6	30	118	107	90	1041	1038	50	0.00	1.02	20	0.01	1.52	30
grid40x40.f10	0.20	3.44	0	0.00	0.04	0	0.00	0.00	80*	0.00	0.04	100*	0.01	0.39	100*
grid40x40.f12	0.00	3.59	0	0.00	0.05	0	0.00	0.00	60*	0.00	0.04	100*	0.00	0.44	100*
grid80x80.f10	45.3	53.3	0	31.2	23.2	10	0.21	0.26	20	0.03	0.10	60	0.25	0.97	80*
grid80x80.f15	4.06	11.8	0	0.00	0.11	0	0.00	0.10	0	0.01	0.54	0	0.08	1.69	0

552 precision ties more gracefully at convergence (iter=100, 500). The two formats report the same
553 underlying trials; layout is under review.

554 B.18 Original v1 Metric (log-scale, for backwards compatibility)

555 The pre-Apr-24 plan reported $\log_{10} |D - D^{LP}|$. This compresses dynamic range and obscures the sign
556 convention (the dual is always above D^{LP} , so the absolute value is redundant). The v2 metric reported
557 in Tables 1, 18, and 19 drops both the absolute value and the outer $\log_{10}$ for direct interpretation.

558 B.19 TTA on MAP: Marginal Benefit

559 Section 4.2 introduces test-time adaptation (TTA) as a general technique. Tables 18–19 demonstrate
560 strong gains from TTA on the CMPE μ -trajectory. We also evaluated TTA on standalone MAP (Ta-

Table 17: **Standalone bound quality (mean $\pm$ std):** dual gap $|D - D^{\text{LP}}|$ at fixed JGLP iterations across 37 PGMs. J = zero-init, N = neural-init. Mean and std (population, ddof=0) computed over 10 bin-0 test seeds per PGM. **Bold** = neural is tie-or-better than J (winning, or within $\max(\sigma_N, \sigma_J)$). Companion to Table 1 (main body) using median + win% format. Layout under review for migration.

PGM	iter=10		iter=50		iter=100		iter=500	
	J	N	J	N	J	N	J	N
BN_12	0.638 $\pm$ 0.238	0.373 $\pm$ 0.179	0.184 $\pm$ 0.147	0.116 $\pm$ 0.105	0.0878 $\pm$ 0.0971	0.0595 $\pm$ 0.0723	0.00828 $\pm$ 0.0170	0.00699 $\pm$ 0.0145
BN_126	0.248 $\pm$ 0.342	0.239 $\pm$ 0.338	0.0647 $\pm$ 0.147	0.0639 $\pm$ 0.146	0.0150 $\pm$ 0.0389	0.0149 $\pm$ 0.0388	1.01e-06 $\pm$ 1.16e-06	1.08e-06 $\pm$ 1.14e-06
BN_13	0.474 $\pm$ 0.115	0.255 $\pm$ 0.0878	0.116 $\pm$ 0.0731	0.0884 $\pm$ 0.0715	0.0584 $\pm$ 0.0525	0.0503 $\pm$ 0.0538	0.00197 $\pm$ 0.00502	0.00263 $\pm$ 0.00556
BN_14	0.278 $\pm$ 0.143	0.174 $\pm$ 0.128	0.0937 $\pm$ 0.137	0.0420 $\pm$ 0.0368	0.0546 $\pm$ 0.143	0.00575 $\pm$ 0.00800	0.0483 $\pm$ 0.145	8.25e-06 $\pm$ 1.59e-05
BN_9	0.162 $\pm$ 0.156	0.0607 $\pm$ 0.0701	0.126 $\pm$ 0.169	0.00680 $\pm$ 0.0188	0.125 $\pm$ 0.170	0.00147 $\pm$ 0.00442	0.125 $\pm$ 0.170	1.35e-07 $\pm$ 6.90e-08
Grids_18	14.3 $\pm$ 5.05	13.7 $\pm$ 4.89	2.01 $\pm$ 0.870	1.96 $\pm$ 0.864	1.07 $\pm$ 0.561	1.06 $\pm$ 0.553	0.0228 $\pm$ 0.0303	0.0228 $\pm$ 0.0303
Grids_19	1.68 $\pm$ 0.984	1.38 $\pm$ 0.814	0.0356 $\pm$ 0.0413	0.0276 $\pm$ 0.0291	0.00756 $\pm$ 0.0148	0.00627 $\pm$ 0.0116	1.60e-06 $\pm$ 2.42e-06	1.48e-06 $\pm$ 1.62e-06
Grids_21	20.8 $\pm$ 6.78	20.0 $\pm$ 6.67	2.88 $\pm$ 1.37	2.83 $\pm$ 1.33	1.60 $\pm$ 0.823	1.59 $\pm$ 0.818	0.0603 $\pm$ 0.117	0.0599 $\pm$ 0.117
Grids_22	6.92 $\pm$ 1.21	6.69 $\pm$ 1.24	0.810 $\pm$ 0.564	0.810 $\pm$ 0.572	0.284 $\pm$ 0.330	0.282 $\pm$ 0.347	0.0147 $\pm$ 0.0299	0.0145 $\pm$ 0.0269
Grids_23	14.6 $\pm$ 2.33	13.7 $\pm$ 2.22	2.56 $\pm$ 0.917	2.48 $\pm$ 0.943	0.904 $\pm$ 0.571	0.901 $\pm$ 0.581	0.0616 $\pm$ 0.120	0.0568 $\pm$ 0.114
Grids_24	47.7 $\pm$ 6.89	46.8 $\pm$ 6.79	9.80 $\pm$ 3.14	9.76 $\pm$ 3.16	4.11 $\pm$ 1.56	4.12 $\pm$ 1.60	0.183 $\pm$ 0.164	0.181 $\pm$ 0.162
Grids_25	4.94 $\pm$ 0.820	3.35 $\pm$ 0.604	0.544 $\pm$ 0.301	0.397 $\pm$ 0.283	0.105 $\pm$ 0.0682	0.0752 $\pm$ 0.0596	7.14e-05 $\pm$ 0.000161	7.14e-05 $\pm$ 0.000162
Grids_26	3.13 $\pm$ 2.19	3.12 $\pm$ 2.17	0.138 $\pm$ 0.147	0.138 $\pm$ 0.147	2.50e-08 $\pm$ 4.90e-08	8.62e-07 $\pm$ 9.57e-07	2.51e-08 $\pm$ 4.86e-08	1.83e-07 $\pm$ 6.33e-08
Grids_27	31.4 $\pm$ 4.33	30.6 $\pm$ 4.22	6.42 $\pm$ 1.84	6.34 $\pm$ 1.84	2.51 $\pm$ 1.06	2.49 $\pm$ 1.08	0.133 $\pm$ 0.134	0.131 $\pm$ 0.131
Grids_28	5.21 $\pm$ 3.22	5.22 $\pm$ 3.19	0.279 $\pm$ 0.383	0.279 $\pm$ 0.380	9.18e-09 $\pm$ 1.30e-08	2.65e-06 $\pm$ 1.92e-06	1.90e-07 $\pm$ 1.35e-07	1.56e-07 $\pm$ 7.86e-08
Grids_29	1.33 $\pm$ 0.849	1.30 $\pm$ 0.851	0.117 $\pm$ 0.115	0.115 $\pm$ 0.114	0.00990 $\pm$ 0.0226	0.00923 $\pm$ 0.0207	2.77e-08 $\pm$ 5.44e-08	2.77e-08 $\pm$ 5.44e-08
Grids_30	14.8 $\pm$ 3.16	12.9 $\pm$ 2.76	5.11 $\pm$ 2.08	4.78 $\pm$ 1.98	2.57 $\pm$ 1.55	2.49 $\pm$ 1.51	0.576 $\pm$ 0.588	0.569 $\pm$ 0.594
Promedax_21	0.948 $\pm$ 1.45	0.266 $\pm$ 0.296	0.948 $\pm$ 1.45	0.00372 $\pm$ 0.000281	0.948 $\pm$ 1.45	0.000406 $\pm$ 3.82e-05	0.948 $\pm$ 1.45	1.45e-05 $\pm$ 2.65e-06
Promedax_32	2.18 $\pm$ 1.79	0.215 $\pm$ 0.129	2.18 $\pm$ 1.79	0.00316 $\pm$ 0.000636	2.18 $\pm$ 1.79	0.000683 $\pm$ 0.000285	2.18 $\pm$ 1.79	1.82e-06 $\pm$ 1.43e-06
Promedax_33	14.8 $\pm$ 1.59	0.343 $\pm$ 0.276	1.85 $\pm$ 1.59	0.0326 $\pm$ 0.00332	1.85 $\pm$ 1.59	0.0138 $\pm$ 0.00147	1.85 $\pm$ 1.59	8.92e-05 $\pm$ 3.12e-06
Promedax_38	3.79e-08 $\pm$ 3.50e-08	0.0555 $\pm$ 0.00182	3.79e-08 $\pm$ 3.50e-08	0.00962 $\pm$ 0.000214	3.80e-08 $\pm$ 3.50e-08	0.00285 $\pm$ 7.78e-05	3.79e-08 $\pm$ 3.50e-08	2.78e-06 $\pm$ 1.19e-07
Promedax_40	0.0391 $\pm$ 0.0479	0.0204 $\pm$ 0.006070	5.64e-07 $\pm$ 8.04e-07	0.00104 $\pm$ 0.000120	5.64e-07 $\pm$ 8.04e-07	0.000133 $\pm$ 2.02e-05	5.64e-07 $\pm$ 8.04e-07	6.50e-07 $\pm$ 4.99e-07
Promedax_44	0.0285 $\pm$ 0.0285	0.0272 $\pm$ 0.00369	6.43e-07 $\pm$ 1.00e-06	0.00239 $\pm$ 0.000430	3.77e-07 $\pm$ 1.07e-06	0.000586 $\pm$ 0.000107	3.77e-07 $\pm$ 1.07e-06	8.07e-07 $\pm$ 7.50e-07
Promedax_51	0.954 $\pm$ 0.477	0.0212 $\pm$ 0.00364	0.954 $\pm$ 0.477	0.00141 $\pm$ 0.000552	0.954 $\pm$ 0.477	0.000204 $\pm$ 0.000106	0.954 $\pm$ 0.477	6.49e-07 $\pm$ 3.41e-08
PF_11	22.2 $\pm$ 3.05	19.4 $\pm$ 3.53	7.54 $\pm$ 2.48	7.10 $\pm$ 2.35	3.84 $\pm$ 1.99	3.69 $\pm$ 1.96	0.576 $\pm$ 0.588	0.570 $\pm$ 0.594
Seg_12	1.19 $\pm$ 0.900	0.701 $\pm$ 0.761	0.177 $\pm$ 0.321	0.0187 $\pm$ 0.0353	0.163 $\pm$ 0.327	1.96e-07 $\pm$ 1.11e-07	0.163 $\pm$ 0.327	1.96e-07 $\pm$ 1.11e-07
Seg_14	0.189 $\pm$ 0.0474	0.0176 $\pm$ 0.0163	2.20e-06 $\pm$ 3.00e-06	0.000147 $\pm$ 0.000247	2.20e-06 $\pm$ 3.03e-06	6.06e-06 $\pm$ 8.42e-06	2.20e-06 $\pm$ 3.03e-06	2.39e-06 $\pm$ 3.05e-06
Seg_16	1.93 $\pm$ 0.396	0.417 $\pm$ 0.273	0.236 $\pm$ 0.202	0.0286 $\pm$ 0.0749	0.0280 $\pm$ 0.0567	7.66e-06 $\pm$ 1.50e-05	1.24e-06 $\pm$ 1.59e-06	1.47e-06 $\pm$ 1.58e-06
Seg_18	0.627 $\pm$ 0.692	0.419 $\pm$ 0.478	0.0731 $\pm$ 0.102	0.0504 $\pm$ 0.0773	0.00506 $\pm$ 0.0152	4.45e-06 $\pm$ 9.49e-06	0.00506 $\pm$ 0.0152	1.94e-06 $\pm$ 2.69e-06
Seg_19	0.125 $\pm$ 0.150	0.104 $\pm$ 0.114	0.0443 $\pm$ 0.123	0.00682 $\pm$ 0.0123	0.0437 $\pm$ 0.123	5.24e-06 $\pm$ 6.52e-06	0.0437 $\pm$ 0.123	1.09e-06 $\pm$ 1.14e-06
grid10 $\times$ 10.15	9.12e-09 $\pm$ 1.17e-08	0.00197 $\pm$ 0.00206	9.12e-09 $\pm$ 1.17e-08	1.22e-05 $\pm$ 1.73e-05	9.12e-09 $\pm$ 1.20e-08	4.60e-08 $\pm$ 2.60e-08	9.10e-09 $\pm$ 1.17e-08	4.61e-08 $\pm$ 2.61e-08
grid20 $\times$ 20.10	14.8 $\pm$ 3.16	12.9 $\pm$ 2.76	5.11 $\pm$ 2.08	4.79 $\pm$ 1.98	2.57 $\pm$ 1.55	2.49 $\pm$ 1.51	0.576 $\pm$ 0.588	0.570 $\pm$ 0.594
grid20 $\times$ 20.15	1.33 $\pm$ 0.849	1.30 $\pm$ 0.850	0.117 $\pm$ 0.115	0.115 $\pm$ 0.114	0.00990 $\pm$ 0.0226	0.00922 $\pm$ 0.0207	2.77e-08 $\pm$ 5.44e-08	1.52e-07 $\pm$ 5.38e-08
grid40 $\times$ 40.10	3.19e-09 $\pm$ 3.72e-09	0.000370 $\pm$ 0.000170	3.19e-09 $\pm$ 3.72e-09	6.82e-07 $\pm$ 1.30e-06	3.19e-09 $\pm$ 3.72e-09	4.60e-08 $\pm$ 1.90e-08	3.20e-09 $\pm$ 3.70e-09	4.62e-08 $\pm$ 1.88e-08
grid40 $\times$ 40.12	3.44e-09 $\pm$ 4.70e-09	0.000500 $\pm$ 0.000300	3.44e-09 $\pm$ 4.70e-09	3.99e-07 $\pm$ 6.75e-07	3.44e-09 $\pm$ 4.70e-09	4.80e-08 $\pm$ 1.30e-08	3.40e-09 $\pm$ 4.70e-09	4.83e-08 $\pm$ 1.33e-08
grid80 $\times$ 80.10	0.469 $\pm$ 0.451	0.344 $\pm$ 0.310	0.159 $\pm$ 0.435	0.0130 $\pm$ 0.0282	0.147 $\pm$ 0.438	0.000721 $\pm$ 0.00184	0.146 $\pm$ 0.438	2.05e-07 $\pm$ 2.80e-07
grid80 $\times$ 80.15	0.0255 $\pm$ 0.0561	0.0262 $\pm$ 0.0560	2.00e-06 $\pm$ 4.30e-06	1.76e-05 $\pm$ 1.52e-05	5.33e-07 $\pm$ 1.09e-06	1.34e-06 $\pm$ 1.36e-06	5.33e-07 $\pm$ 1.09e-06	6.86e-07 $\pm$ 1.10e-06

Table 18: **CMPE bound quality (mean $\pm$ std):** $|L(\mu) - L^{\text{LP}}(\mu)|$ at fixed inner JGLP iterations, $K = 1$ constraint. M1=zero-init, M2=neural warm-start, M3=neural+replay-buffer TTA. Mean and std (ddof=0) computed across (seed $\times$ μ -grad step) trials. **Bold** = neural tie-or-better than M1 (winning or within $\max(\sigma_M, \sigma_1)$). Companion to Table 18 (main body) using median + win% format. Layout under review for migration.

PGM	iter=10				iter=50				iter=100				iter=500			
	M1	M2	M3	std	M1	M2	M3	std	M1	M2	M3	std	M1	M2	M3	std
BN_12	0.760 ± 0.301	0.435 ± 0.237	0.0633 ± 0.144	0.256 ± 0.420	0.124 ± 0.0978	0.0226 ± 0.0456	0.300 ± 0.433	0.0598 ± 0.0534	0.0135 ± 0.0261	0.162 ± 0.443	0.0135 ± 0.0143	0.00377 ± 0.00177	0.162 ± 0.443	0.00953 ± 0.00297	0.000223 ± 0.00193	0.162 ± 0.443
BN_13	0.465 ± 0.186	0.426 ± 0.175	0.0407 ± 0.0988	0.108 ± 0.0741	0.0990 ± 0.0697	0.0108 ± 0.0314	0.0206 ± 0.0300	0.0188 ± 0.0276	0.00386 ± 0.0128	0.00101 ± 0.00314	0.000953 ± 0.00297	0.000223 ± 0.00193	0.00101 ± 0.00314	0.000953 ± 0.00297	0.000223 ± 0.00193	0.00101 ± 0.00314
BN_14	0.533 ± 0.142	0.466 ± 0.137	0.0387 ± 0.0991	0.145 ± 0.0626	0.131 ± 0.0602	0.0123 ± 0.0361	0.0410 ± 0.0418	0.0358 ± 0.0397	0.00361 ± 0.0158	0.000117 ± 0.00117	0.000180 ± 0.00120	6.95e-05 ± 0.00114	0.000117 ± 0.00117	0.000180 ± 0.00120	6.95e-05 ± 0.00114	0.000117 ± 0.00117
BN_9	0.135 ± 0.0978	0.111 ± 0.0986	0.0131 ± 0.0388	0.058 ± 0.0604	0.0385 ± 0.0409	0.00303 ± 0.0154	0.0488 ± 0.0644	0.0122 ± 0.0262	0.00145 ± 0.00748	0.0036 ± 0.0059	0.000115 ± 0.00070	1.24e-05 ± 0.000213	0.0036 ± 0.0059	0.000115 ± 0.00070	1.24e-05 ± 0.000213	0.0036 ± 0.0059
Grids_26	15.6 ± 23.8	15.8 ± 23.8	4.76 ± 14.1	1.36 ± 4.08	1.36 ± 4.08	0.533 ± 2.51	0.0554 ± 0.172	0.0554 ± 0.172	0.00991 ± 0.00606	8.08e-06 ± 3.30e-05	8.21e-06 ± 3.50e-05	3.61e-06 ± 1.15e-05	8.08e-06 ± 3.30e-05	8.21e-06 ± 3.50e-05	3.61e-06 ± 1.15e-05	8.08e-06 ± 3.30e-05
Grids_28	38.8 ± 59.2	38.5 ± 59.2	19.2 ± 42.4	2.26 ± 7.62	2.26 ± 7.62	1.16 ± 4.08	0.0177 ± 0.0598	0.0140 ± 0.0469	0.00194 ± 0.0106	1.35e-07 ± 3.95e-07	3.02e-07 ± 3.96e-07	4.29e-08 ± 5.87e-08	1.35e-07 ± 3.95e-07	3.02e-07 ± 3.96e-07	4.29e-08 ± 5.87e-08	1.35e-07 ± 3.95e-07
Grids_29	4.03 ± 4.68	4.03 ± 4.68	0.507 ± 0.507	0.115 ± 0.358	0.115 ± 0.358	0.0554 ± 0.172	0.00177 ± 0.0598	0.00140 ± 0.0469	0.000194 ± 0.0106	1.35e-07 ± 3.95e-07	3.02e-07 ± 3.96e-07	4.29e-08 ± 5.87e-08	1.35e-07 ± 3.95e-07	3.02e-07 ± 3.96e-07	4.29e-08 ± 5.87e-08	1.35e-07 ± 3.95e-07
Grids_30	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_31	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_32	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_33	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_34	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_35	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_36	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_37	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_38	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_39	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_40	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_41	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_42	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_43	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_44	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_45	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_46	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_47	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_48	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_49	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_50	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_51	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_52	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_53	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_54	0.367 ± 0.338	0.353 ± 0.631	0.0100 ± 0.141	0.0211 ± 0.0466	0.0226 ± 0.143	0.00446 ± 0.0227	0.00334 ± 0.149	0.00364 ± 0.0399	0.000583 ± 0.00552	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063	0.278e-07 ± 3.77e-07	3.21e-07 ± 3.78e-07	0.000662 ± 0.00063
Grids_55	0.367 ± 0.338	0.353 ± 0.631	0													

Table 19: **CMPE bound quality (mean $\pm$ std): $|L(\mu) - L^{LP}(\mu)|$** at fixed inner JGLP iterations, $K = 2$ constraints. M1=zero-init, M2=neural warm-start, M3=neural+replay-buffer TTA. Mean and std (ddof=0) computed across (seed $\times$ μ -grad step) trials. **Bold** = neural tie-or-better than M1 (winning or within $\max(\sigma_M, \sigma_1)$). Companion to Table 19 (main body) using median + win% format. Layout under review for migration.

PGM	iter=0			iter=50			iter=100			iter=500		
	M1	M2	M3	M1	M2	M3	M1	M2	M3	M1	M2	M3
BN_12	0.658 $\pm$ 0.149	0.646 $\pm$ 0.149	0.0789 $\pm$ 0.132	0.222 $\pm$ 0.0721	0.218 $\pm$ 0.0664	0.0368 $\pm$ 0.0543	0.111 $\pm$ 0.0600	0.109 $\pm$ 0.0506	0.0228 $\pm$ 0.0325	0.0225 $\pm$ 0.0404	0.0215 $\pm$ 0.0202	0.00537 $\pm$ 0.0100
BN_13	0.488 $\pm$ 0.163	0.436 $\pm$ 0.0972	0.0669 $\pm$ 0.113	0.179 $\pm$ 0.240	0.0968 $\pm$ 0.0539	0.0188 $\pm$ 0.0372	0.125 $\pm$ 0.225	0.0363 $\pm$ 0.0351	0.0051 $\pm$ 0.0173	0.0081 $\pm$ 0.264	7.22e-05 $\pm$ 0.00434	5.30e-05 $\pm$ 0.000391
BN_14	0.352 $\pm$ 0.0832	0.313 $\pm$ 0.0058	0.0483 $\pm$ 0.0960	0.0492 $\pm$ 0.0577	0.0409 $\pm$ 0.0363	0.00065 $\pm$ 0.0255	0.0172 $\pm$ 0.0520	0.00970 $\pm$ 0.0138	0.00194 $\pm$ 0.00946	0.00772 $\pm$ 0.0515	3.14e-05 $\pm$ 0.000451	3.10e-05 $\pm$ 0.000451
BN_9	0.190 $\pm$ 0.0828	0.161 $\pm$ 0.0035	0.0283 $\pm$ 0.0563	0.101 $\pm$ 0.0928	0.0433 $\pm$ 0.0489	0.00694 $\pm$ 0.0216	0.0723 $\pm$ 0.0999	0.0109 $\pm$ 0.0226	0.00129 $\pm$ 0.00625	0.0013 $\pm$ 0.104	2.66e-07 $\pm$ 3.24e-07	2.93e-07 $\pm$ 3.20e-07
Grids_26	49.6 $\pm$ 65.7	49.6 $\pm$ 65.7	39.5 $\pm$ 55.9	3.60 $\pm$ 7.09	3.60 $\pm$ 7.09	2.86 $\pm$ 6.37	0.124 $\pm$ 0.303	0.124 $\pm$ 0.303	0.0553 $\pm$ 0.176	2.44e-05 $\pm$ 7.31e-05	2.45e-05 $\pm$ 7.30e-05	1.31e-05 $\pm$ 4.22e-05
Grids_28	109 $\pm$ 140	109 $\pm$ 140	97.4 $\pm$ 129	6.95 $\pm$ 16.2	6.95 $\pm$ 16.2	6.37 $\pm$ 15.8	0.871 $\pm$ 2.37	0.871 $\pm$ 2.37	0.838 $\pm$ 2.32	1.96e-05 $\pm$ 5.87e-05	1.97e-05 $\pm$ 5.87e-05	1.84e-05 $\pm$ 5.60e-05
Grids_29	12.9 $\pm$ 18.0	12.9 $\pm$ 18.0	6.37 $\pm$ 12.0	1.54 $\pm$ 2.43	1.54 $\pm$ 2.43	0.682 $\pm$ 1.57	0.0604 $\pm$ 0.173	0.0604 $\pm$ 0.173	0.0137 $\pm$ 0.0761	1.38e-05 $\pm$ 4.11e-05	1.39e-05 $\pm$ 4.10e-05	4.46e-06 $\pm$ 1.80e-05
Seg_12	0.592 $\pm$ 0.505	0.297 $\pm$ 0.508	0.177 $\pm$ 0.332	0.0468 $\pm$ 0.0850	0.0459 $\pm$ 0.0833	0.0146 $\pm$ 0.0482	0.0152 $\pm$ 0.0303	0.00148 $\pm$ 0.00291	0.000535 $\pm$ 0.00162	3.00e-07 $\pm$ 6.77e-07	3.00e-07 $\pm$ 6.80e-07	4.42e-07 $\pm$ 8.97e-07
Seg_14	0.712 $\pm$ 0.879	0.620 $\pm$ 0.756	0.0976 $\pm$ 0.321	0.00472 $\pm$ 0.0118	0.00328 $\pm$ 0.00860	0.000529 $\pm$ 0.00297	2.03e-05 $\pm$ 7.47e-05	1.96e-05 $\pm$ 6.46e-05	5.40e-06 $\pm$ 2.37e-05	1.57e-07 $\pm$ 1.84e-07	2.71e-07 $\pm$ 1.97e-07	3.53e-07 $\pm$ 1.95e-07
Seg_16	2.37 $\pm$ 2.06	2.05 $\pm$ 1.78	0.477 $\pm$ 0.980	0.234 $\pm$ 0.325	0.207 $\pm$ 0.280	0.0529 $\pm$ 0.143	0.0226 $\pm$ 0.0495	0.0186 $\pm$ 0.0408	0.00368 $\pm$ 0.0171	1.94e-07 $\pm$ 3.55e-07	3.10e-07 $\pm$ 3.81e-07	4.42e-07 $\pm$ 4.21e-07
Seg_18	0.745 $\pm$ 0.765	0.731 $\pm$ 0.730	0.149 $\pm$ 0.330	0.0296 $\pm$ 0.0588	0.0195 $\pm$ 0.0502	0.00365 $\pm$ 0.0236	0.0323 $\pm$ 0.0124	3.96e-05 $\pm$ 7.71e-05	5.83e-05 $\pm$ 0.000229	0.00334 $\pm$ 0.0124	5.31e-07 $\pm$ 4.84e-07	6.20e-07 $\pm$ 4.69e-07
Seg_19	2.81 $\pm$ 3.83	2.54 $\pm$ 3.53	0.889 $\pm$ 1.85	0.917 $\pm$ 1.36	0.841 $\pm$ 1.31	0.415 $\pm$ 0.860	0.436 $\pm$ 0.837	0.393 $\pm$ 0.808	0.251 $\pm$ 0.579	0.0668 $\pm$ 0.171	0.0403 $\pm$ 0.143	0.0259 $\pm$ 0.100
grid10 $\times$ 10.f5	0.434 $\pm$ 0.397	0.182 $\pm$ 0.213	0.0376 $\pm$ 0.118	0.406 $\pm$ 0.420	1.90e-05 $\pm$ 3.62e-05	2.72e-05 $\pm$ 7.54e-05	0.406 $\pm$ 0.420	2.75e-07 $\pm$ 5.64e-07	5.05e-07 $\pm$ 1.30e-06	0.406 $\pm$ 0.420	5.45e-08 $\pm$ 3.11e-08	7.67e-08 $\pm$ 4.40e-08
grid20 $\times$ 20.f5	12.9 $\pm$ 18.0	12.9 $\pm$ 18.0	6.25 $\pm$ 11.9	1.54 $\pm$ 2.43	1.54 $\pm$ 2.43	0.664 $\pm$ 1.56	0.0604 $\pm$ 0.173	0.0604 $\pm$ 0.173	0.0131 $\pm$ 0.0745	1.38e-05 $\pm$ 4.11e-05	1.39e-05 $\pm$ 4.10e-05	4.45e-06 $\pm$ 1.76e-05

Table 20: [v1 metric] MAP on 37 benchmark PGMs: $\log_{10} |D - D^{LP}|$ at fixed JGLP iterations (bin 0, 10 seeds). D^{LP} = exact LP optimum via CVXPY/CLARABEL. J = zero-init JGLP, N = neural-init JGLP. Lower = closer to LP. Last column: % of seeds where neural log-error is smaller than zero at iter=10. Top 15 PGMs by win@10 shown; appendix Table 16 lists the v2 metric for all 37.

PGM	D^{LP}	iter=0		iter=10		iter=50		iter=100		conv		win@10
		J	N	J	N	J	N	J	N	J	N	
BN_12	-31.64	-0.02	-0.17	-0.22	-0.48	-0.74	-1.05	-1.65	-2.04	-6.79	-6.77	100%
BN_13	-44.93	-0.04	-0.19	-0.29	-0.60	-0.97	-1.15	-1.34	-1.55	-7.17	-6.98	100%
BN_14	-38.83	-0.34	-0.39	-0.66	-0.83	-1.23	-1.33	-4.22	-4.52	-6.49	-6.42	100%
Seg_12	-114.24	0.21	-0.05	-0.01	-1.01	-4.13	-4.94	-7.10	-6.69	-7.10	-6.69	100%
Seg_14	-136.64	0.18	-0.42	-0.73	-2.00	-6.34	-4.62	-6.34	-6.13	-6.34	-6.19	100%
Seg_16	-132.72	0.58	0.13	0.29	-0.49	-0.67	-3.88	-6.07	-5.71	-6.23	-6.10	100%
PF_11	4145	1.90	1.70	1.34	1.29	0.85	0.84	0.55	0.51	-0.40	-0.40	100%
Grids_19	2327	1.02	1.01	0.23	0.13	-1.87	-1.86	-6.19	-4.34	-6.41	-6.23	100%
Grids_25	2332	1.15	1.10	0.70	0.53	-0.26	-0.46	-0.99	-1.17	-4.90	-4.91	100%
Grids_30	2767	1.71	1.50	1.17	1.11	0.66	0.62	0.42	0.38	-0.41	-0.40	100%
grid20 $\times$ 20.f10	2767	1.71	1.51	1.17	1.11	0.66	0.62	0.42	0.38	-0.41	-0.40	100%
grid20 $\times$ 20.f5	1322	0.89	0.87	0.07	0.03	-1.05	-1.05	-8.79	-6.00	-9.24	-6.82	90%
Seg_18	-111.99	-0.24	-0.06	-0.66	-0.87	-3.35	-3.79	-6.02	-6.15	-6.02	-6.15	80%
BN_9	-39.98	-0.59	-0.60	-0.97	-1.53	-2.71	-5.77	-6.68	-6.94	-6.68	-6.98	70%
Seg_19	-93.60	-0.41	-0.24	-3.42	-1.16	-5.56	-3.97	-5.71	-5.62	-6.25	-6.60	33%
Degenerate (MBE tight at LP; neural provides no benefit):												
grid10 $\times$ 10	335.89	-8.78	-0.83	-8.78	-3.04	-8.78	-7.35	-8.78	-7.35	-8.78	-7.35	0%
grid40 $\times$ 40.f2	1478	-8.88	-0.44	-9.34	-3.33	-9.34	-7.06	-9.34	-7.36	-9.34	-7.36	0%

576 B.21 LP-Supervised CMPE Pipeline (Alternative)

577 The plan migrates the central narrative to the no-LP pipeline above (Tables 18–19 use it). The
 578 LP-supervised variant uses a hybrid loss $\mathcal{L} = c \cdot \text{MSE}(\mathbf{m}, \mathbf{m}^{LP}) + (1 - c)D_\tau$ with $c = 1.0$ (pure
 579 MSE against the LP-optimal edge message), and remains a viable alternative when an LP solver
 580 is already available in the user’s pipeline. Empirically, it produces slightly tighter raw NN bounds
 581 at iter=0 on degenerate PGMs but is matched or exceeded by the no-LP variant on non-degenerate
 582 PGMs (Table 22).

583 **Settings split in Table 19 (K=2).** All K=1 results in Table 18 use the full setting of 100 μ -grad steps
 584 $\times$ 10 test seeds. For K=2, only BN_12 uses the full 100 $\times$ 10 setting (preserved from a long-running
 585 eval that completed before the fallback); the remaining 13 PGMs use 50 μ -grad steps $\times$ 5 seeds. The
 586 reduction was forced by per-PGM eval cost: at full settings, K=2 grids and segs (and a retry on the
 587 BN family) ran 2.5+ hours per PGM with no first-seed completion, hitting the kill threshold. The
 588 reduced setting still computes 250 (seed, μ -step) trials per PGM (vs. 1000 at full), which is sufficient
 589 for the win-rate confidence intervals reported.

Table 21: TTA on MAP across 12 PGMs (Phase 3 partial; the run was aborted after stalling on Promedas/ProteinFolding). win N = neural-init tie-or-better than zero-init; win A = neural-init + replay-buffer adaptation tie-or-better than zero-init. Pattern is consistent across BN and grid families: TTA on independent MAP queries provides marginal-to-no benefit, frequently slightly worse than plain N at later iters; the replay buffer accumulates messages from prior evidence configurations that do not transfer cleanly. This confirms TTA’s value lies in repeated-query settings (μ -trajectory in CMPE Tables 18–19, Section 4.2), not on independent MAP queries.

PGM	iter=10		iter=50		iter=100		iter=500	
	win N	win A	win N	win A	win N	win A	win N	win A
BN_126	89	52	56	56	78	44	78	56
BN_12	100	100	90	97	100	93	100	100
BN_13	100	100	100	100	60	73	80	77
BN_14	100	100	80	80	80	77	90	90
BN_9	70	67	70	60	100	100	100	100
grid10×10.f5	40	30	40	40	70	63	70	70
grid20×20.f10	100	100	80	83	70	77	60	63
grid20×20.f5	90	77	50	43	20	20	30	17
grid40×40.f10	0	0	80	47	100	93	100	100
grid40×40.f2	0	0	60	23	100	93	100	100
grid80×80.f10	10	10	20	10	60	33	80	60
grid80×80.f5	0	3	0	0	0	0	0	0

Table 22: No-LP vs LP-supervised CMPE pipeline at iter=10 (K=1, 5 test seeds × 50 μ -grad steps). Δ columns are no-LP minus LP-supervised in percentage points.

PGM	no-LP M2	no-LP M3	main M2	main M3	Δ M2	Δ M3	note
BN_9	97.6	95.2	75.2	92.8	+22.4	+2.4	
BN_14	100	100	100	99.2	± 0	+0.8	
Segmentation_18	78.7	92.4	52.6	89.6	+26.1	+2.8	
Segmentation_19	46.4	80.0	7.2	72.0	+39.2	+8.0	
grid10×10	10.0	10.0	10.0	10.0	± 0	± 0	degenerate
grid40×40.f10	0.0	0.0	0.0	0.0	± 0	± 0	degenerate

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Section 1 (Contributions) state the four claims — deterministic validity by construction, self-supervised training without LP labels, multi-constraint CMPE extension, and empirical gains on UAI benchmarks — all of which are supported by Section 4 and the appendix.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Limitations are discussed inline: degenerate i-bound cases where MBE already attains the LP optimum (Sec. 4.1, Sec. 9.15), per-graph training requirement (Sec. 6), out-of-distribution μ values for CMPE (Sec. 4.2), ordinal bias of scalar evidence encoding (Sec. 9.17), and tightness only when the join-graph LP relaxation is itself tight (Sec. 3, end).

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The single theoretical claim (Proposition 7: $D_\tau \geq D \geq \text{MAP}^*$) is stated in Sec. 3 with a one-line proof from convexity of LSE; the antisymmetric construction guaranteeing validity is given in Eqs. (4)–(6).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Architecture (3-layer MLP, 512/512/256), optimizer (AdamW, cosine schedule, $lr = 10^{-3}$, 200 epochs, batch 64), τ schedule, evidence sampling, training data construction, replay-buffer settings, and JGLP iteration budgets are fully specified in Sec. 4 and Sec. 9.8–9.16.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Anonymized code is provided in supplementary material; UAI 2022 benchmark PGMs are publicly available from the UAI competition website; CVXPY/CLARABEL is open-source.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Sec. 4 specifies i-bound 4, 10 seeds per (PGM, evidence-bin), evidence ratios, μ -grid, σ for CMPE; Sec. 9.8–9.10 give CMPE-specific settings; Sec. 9.16 reports an 11-variant ablation suite.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All headline tables report mean $\pm$ population standard deviation over 10 bin-0 test seeds per PGM (MAP) or 10 seeds $\times$ 100 μ -grad steps (CMPE); Sec. 9.18 reports seed-subset stability ($\pm 4pp$).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Wall-clock timings reported in Tables and neural forward pass ~ 0.04 ms; JGLP iteration costs 1.6–131 ms depending on PGM. Hardware: single NVIDIA GPU; CVXPY/CLARABEL on CPU.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work conforms with the NeurIPS Code of Ethics; it uses only public benchmark PGMs and involves no human subjects, scraped data, or deployed systems.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This is foundational work on dual-bound computation for graphical-model inference. There is no direct path to deployment in a setting with foreseeable positive or negative societal impact distinct from PGM inference research generally.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released artifact is a per-PGM neural warm-start network for dual bounds; it has no generative or dual-use risk.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: UAI 2022 benchmark PGMs are cited and used under the competition’s public-release terms; CVXPY (Apache 2.0) and CLARABEL (Apache 2.0) are cited; PyTorch (BSD-3) is the training framework.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Trained network checkpoints, training scripts, and evaluation harness are released in the supplementary material with a README and license.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No human subjects or crowdsourcing.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

905 Question: Does the paper describe potential risks incurred by study participants, whether
906 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
907 approvals (or an equivalent approval/review based on the requirements of your country or
908 institution) were obtained?

909 Answer: [N/A]

910 Justification: No human subjects research.

911 Guidelines:

- 912 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
913 with human subjects.
- 914 • Depending on the country in which research is conducted, IRB approval (or equivalent)
915 may be required for any human subjects research. If you obtained IRB approval, you
916 should clearly state this in the paper.
- 917 • We recognize that the procedures for this may vary significantly between institutions
918 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
919 guidelines for their institution.
- 920 • For initial submissions, do not include any information that would break anonymity (if
921 applicable), such as the institution conducting the review.

922 16. Declaration of LLM usage

923 Question: Does the paper describe the usage of LLMs if it is an important, original, or
924 non-standard component of the core methods in this research? Note that if the LLM is used
925 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
926 scientific rigor, or originality of the research, declaration is not required.

927 Answer: [N/A]

928 Justification: LLMs were used only for writing, editing, and formatting; they do not form a
929 component of the core methodology.

930 Guidelines:

- 931 • The answer [N/A] means that the core method development in this research does not
932 involve LLMs as any important, original, or non-standard components.
- 933 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
934 be described.